

Σavante

**Arquitecturas de Sistemas.
Arquitectura Cliente/Servidor
y Multicapas. Arquitectura
de Servicios web y protocolos
asociados**

ÍNDICE

1. Arquitecturas de sistemas	5
1.1. Tipos de arquitecturas más comunes	7
1.2. Interoperabilidad entre sistemas	12
1.2.1. Definición y objetivos	12
1.2.2. Niveles de Interoperabilidad	12
1.2.3. Protocolos y estándares comunes	13
1.2.4. Casos de uso	15
2. Tipos de sistemas	16
2.1. Tipo: sistemas distribuidos	17
2.1.1. Características de los sistemas distribuidos	19
2.1.2. Propiedades de los sistemas distribuidos	19
2.1.2.1. Transparencia	19
2.1.2.2. Escalabilidad	21
2.1.2.3. Fiabilidad y Tolerancia a fallos	22
2.1.2.4. Consistencia	23
2.1.3. Aplicaciones distribuidas	24
2.1.3.1. Objetivos de las Aplicaciones Distribuidas	24
3. Arquitectura cliente/servidor	29
3.1. Características	30
3.2. Tipos de comunicación	32
3.3. Funcionamiento	33
3.4. Componentes	34
3.4.1. Cliente	35
3.4.2. Servidor	35
3.4.3. <i>Middleware</i>	37
3.5. Tipos de arquitecturas cliente/servidor	38
3.5.1. Por el tamaño de los componentes	39
3.5.1.1. Fat Client (Thin Server)	39
3.5.1.2. Fat Server (Thin Client)	39

3.5.2. Por la naturaleza del servicio proporcionado	40
3.5.2.1. Servidores de ficheros	40
3.5.2.2. Servidores de bases de datos	41
3.5.2.3. Servidores de transacciones	42
3.5.2.4. Servidores de objetos	42
3.5.2.5. Servidores web	43
3.5.3. Patrón de Diseño MVC	43
3.6. Modelos cliente/servidor	45
3.6.1. A nivel de Hardware	47
3.6.2. A nivel de Software	48
3.6.2.1. Modelo de dos capas	49
3.6.2.2. Modelo de tres capas	52
3.6.2.3. Modelo de N capas (N-Layer)	55
4. Arquitecturas de servicios web	56
4.1. Servicios web	58
4.2. Protocolos Web (Web Services Protocol Stack)	61
4.2.1. XML (Extensible Markup Language)	62
4.2.2. SOAP	63
4.2.2.1. Características	64
4.2.2.2. Mensajes SOAP	65
4.2.2.3. Estructura del mensaje orientado al documento	66
4.2.3. UDDI (Universal Description, Discovery and Integration)	69
4.2.4. WSIL	71
4.2.5. WSDL (Web Services Description Language)	72
4.2.6. Estándares de Seguridad y Gestión Avanzada en Servicios Web	75
4.2.7. REST (Representational State Transfer)	77
4.2.7.1. RESTful	77
4.2.7.2. API REST	78
4.2.7.3. Documentación de APIs REST	80
4.2.7.3.1. RAML (RESTful API Modeling Language)	80
4.2.7.3.2. OpenAPI / Swagger	81

4.2.8. Otros protocolos	81
4.2.8.1. Protocolos genéricos para transmisión de XML	81
4.2.8.2. Protocolos especializados en XML	82
5. Arquitectura SOA	82
6. Bibliografía	87



1. Arquitecturas de sistemas

En los inicios de la programación, la dificultad que entrañaba para la mayoría de las personas, se consideraba un arte. Con el tiempo ha ido evolucionando, descubriendo y desarrollando guías generales, para resolver los problemas, a las que se les ha denominado arquitectura de software, debido a que indican la estructura, funcionamiento e interacción entre las partes del software (a semejanza de los planos de un edificio o construcción).

En el libro "An Introduction to Software Architecture", sus autores David Garlan y Mary Shaw definen que la arquitectura es un nivel de diseño que hace foco en aspectos "más allá de los algoritmos y estructuras de datos de la computación; el diseño y especificación de la estructura global del sistema es un nuevo tipo de problema".



+ Info

El concepto de arquitectura de software se hizo popular en los años 1990 tras reconocerse la denominada crisis del software y como tema de interés de la incipiente disciplina de la ingeniería del software.

Aunque en los años 1960 ya se acercaba el concepto de arquitectura de software en los círculos de investigación (por ejemplo, por Edsger Dijkstra).

Arquitectura a nivel de software

La arquitectura a nivel de software (también denominada arquitectura lógica) es el diseño de más alto nivel de la estructura de un sistema, y:

- Consiste en un conjunto de patrones y abstracciones coherentes que proporcionan un marco definido y claro para interactuar con el código fuente del software.
- Se selecciona y diseña con base en objetivos (requisitos) y restricciones.
- Los objetivos son aquellos prefijados para el sistema de información, pero no solamente los de tipo funcional, también otros objetivos como el mantenimiento, la auditoría, flexibilidad e interacción con otros sistemas de información.
- Las restricciones son aquellas limitaciones derivadas de las tecnologías disponibles para implementar sistemas de información.



- Unas arquitecturas son más recomendables de implementar con ciertas tecnologías mientras que otras tecnologías no son aptas para determinadas arquitecturas.

Por ejemplo, no es viable emplear una arquitectura de software de tres capas para implementar sistemas en tiempo real.

- Define, de manera abstracta, los componentes que llevan a cabo alguna tarea de computación, sus interfaces y la comunicación entre ellos.



Atención

Toda arquitectura debe ser implementable en una arquitectura física, que consiste simplemente en determinar qué computadora tendrá asignada cada tarea.

Modelos o vistas

Para describir de una manera más comprensible cada uno de los diversos aspectos del software, se utilizan modelos o vistas, cada uno de los cuales constituye una descripción parcial de una misma arquitectura y es deseable que exista cierto solapamiento entre ellos, deben ser coherentes entre sí puesto que describen la misma cosa.

Dependiendo del paradigma de desarrollo, son necesarios un diferente número y tipo de vistas o modelos para describir una arquitectura, pero como mínimo hay tres vistas que son fundamentales en cualquier arquitectura:

- La visión **estática**: describe qué componentes tiene la arquitectura.
- La visión **funcional**: describe qué hace cada componente.
- La visión **dinámica**: describe cómo se comportan los componentes a lo largo del tiempo y cómo interactúan entre sí.

Las vistas o modelos de una arquitectura de software pueden expresarse mediante uno o varios lenguajes (diagramas de estado, los diagramas de flujo de datos, etc.).

Existe cierto consenso en adoptar UML (Unified Modeling Language, lenguaje unificado de modelado) como lenguaje único para todos los modelos o vistas. Pero surge el problema de que un lenguaje generalista no sea capaz de describir determinadas restricciones de un sistema de información (o expresarlas de manera comprensible).



Recuerda ver las clases emitidas en Temario Audiovisual

Las clases impartidas en directo y disponibles en Campus, en Temario Audiovisual, te ayudarán al entendimiento de la unidad, y además pueden tener información adicional.

[ACCEDE DIRECTAMENTE DESDE AQUÍ](#)

1.1. Tipos de arquitecturas más comunes

En función de las ventajas e inconvenientes de cada arquitectura conocida (se podría "inventar" una nueva arquitectura de software para cada sistema de información).

Las arquitecturas más universales son:

- **Arquitectura Monolítica.**

El desarrollo de software empezó utilizando una arquitectura monolítica que agrupaba todas sus funciones y servicios dentro de una base única y centralizada de código.

Este tipo de arquitectura se caracteriza por:

- Los programas son fáciles de desarrollar.
- El despliegue y la ejecución del software son muy sencillos.
- El costo de desarrollo es bajo en comparación con otras arquitecturas.

Este tipo de arquitectura ofrece ciertos problemas como la escalabilidad o la dificultad para los desarrolladores (necesitan entender todo el código de la aplicación), lo cual ha producido que haya ido quedando desfasada, sobre todo al crecer los proyectos en complejidad, número de desarrolladores, usuarios y cargas de trabajo, aunque su sencillez y bajo coste hace que siga siendo interesante para ciertos proyectos con bajos requerimientos.

- **Descomposición Modular.**

El software se estructura en grupos funcionales muy acoplados.

El diseño estructurado persigue elaborar algoritmos que cumplan la propiedad de modularidad. Para ello, dado un problema que se pretende resolver mediante la elaboración de un programa de ordenador, se busca dividir dicho programa en módulos siguiendo los principios de diseño de descomposición por refinamientos sucesivos, creación de una jerarquía modular y elaboración de módulos independientes.



- **Sistemas Distribuidos.**

En este tipo de arquitectura no se etiqueta un cliente o un servidor, el sistema debe verse como un conjunto de objetos, que tienen como características que una parte de los objetos proporcionan una interfaz a un conjunto de servicios.

Se logra una mayor velocidad en el procesamiento, por ejemplo, al consultar una base de datos, los procedimientos se dividen entre los distintos nodos, obteniendo una respuesta mucho más rápida que si se realiza con un único nodo.

- **Cliente-servidor.**

Las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y a los demandantes de estos servicios se le llama clientes.

- **En pipeline.**

La arquitectura en pipeline (basada en filtros) consiste en ir transformando un flujo de datos en un proceso comprendido por varias fases secuenciales, siendo la entrada de cada una la salida de la anterior.

El pipeline es una técnica para implementar simultaneidad a nivel de instrucciones dentro de un solo procesador.

Pipelining intenta mantener ocupada a cada parte del procesador, dividiendo las instrucciones entrantes en una serie de pasos secuenciales, que se realizan por diferentes unidades del procesador que trabajan de forma simultánea. Aumenta el rendimiento de la CPU a una velocidad de reloj determinada, aunque puede aumentar la latencia debido a la sobrecarga adicional del proceso de pipeline en sí.

Esta arquitectura es muy común en el desarrollo de programas para el intérprete de comandos, ya que se pueden conectar comandos fácilmente con tuberías (pipe), y también se utiliza en el paradigma de programación funcional, ya que equivale a la composición de funciones matemáticas.

- **'Peer to Peer', P2P (Entre pares).**

Es un tipo de arquitectura para la comunicación entre aplicaciones que permite a individuos comunicarse y compartir información con otros individuos sin necesidad de un servidor central que facilite la comunicación.

Se trata del uso de red de ordenadores en la que todos o algunos aspectos funcionan sin clientes ni servidores fijos, sino con una serie de nodos que se comportan como iguales entre sí. Pueden actuar simultáneamente como clientes y servidores respecto a los demás nodos de la red.

Las redes P2P permiten el intercambio directo de información, en cualquier formato, entre los ordenadores interconectados.



- **En pizarra.**

Es un modelo arquitectónico de software habitualmente utilizado en sistemas expertos, sistemas multiagente y, en general, sistemas basados en el conocimiento.

Consta de múltiples elementos funcionales, denominados agentes, y un instrumento de control denominado pizarra.

Los agentes suelen estar especializados en una tarea concreta o elemental, y todos ellos cooperan para alcanzar una meta común, si bien, sus objetivos individuales no están aparentemente coordinados.

El comportamiento básico de cualquier agente consiste en examinar la pizarra, realizar su tarea y escribir sus conclusiones en la misma pizarra. De esta manera, otro agente puede trabajar sobre los resultados generados por otro.

El estado inicial de la pizarra es una descripción del problema que resolver y el estado final será la solución del problema.

La pizarra tiene un doble papel:

- Coordina a los distintos agentes.
- Facilita su intercomunicación.

La computación termina cuando se alcanza alguna condición deseada entre los resultados escritos en la pizarra.

Los resultados generados por los agentes deben responder a un lenguaje y semántica común, generalmente se utilizan formalismos lógicos o matemáticos, tales como expresiones lógicas de primer orden.

- **Arquitectura CORBA.**

Common Object Request Broker Architecture (CORBA) es un estándar definido por Object Management Group (OMG) que permite que diversos componentes de software escritos en múltiples lenguajes de programación y que corren en diferentes computadoras, puedan trabajar juntos; es decir, facilita el desarrollo de aplicaciones distribuidas en entornos heterogéneos.

Fue el primer producto propuesto por OMG. Su objetivo es ayudar a reducir la complejidad, disminuir los costes y acelerar la introducción de nuevas aplicaciones informáticas, promoviendo la teoría y la práctica de la tecnología de objetos en los sistemas distribuidos.

Es una tecnología que oculta la programación a bajo nivel de aplicaciones distribuidas, que también brinda al programador una tecnología orientada a objetos (las funciones objetos y estos objetos pueden estar en diferentes máquinas, pero el programador accede a ellos a través de funciones normales dentro de su programa).



CORBA es más que una especificación multiplataforma, también define servicios habitualmente necesarios como seguridad y transacciones. Y así este no es un sistema operativo en sí, en realidad es un middleware.

Su primera versión se lanzó en 1991, y en 1995 aparece CORBA 2, que permite que puedan cooperar implementaciones de diferentes fabricantes, que pueda ser implementado sobre cualquier nivel de transporte y que pueda funcionar en Internet sobre TCP/IP, creando un protocolo: IIOP (Internet IOP).

CORBA 3 se lanza en 2002, donde se introduce, entre otras cosas el CORBA Component Model (CCM), con el que se pasó de un modelo de objetos distribuidos (EJB, restringido a Java) a un modelo distribuido orientado a componentes. Esta versión 3 se lanza como intento de competir con Microsoft y su modelo de programación de objetos distribuidos DCOM.

El Modelo de Objetos de Componentes Distribuidos DCOM, (Distributed Component Object Model,) es una tecnología propietaria de Microsoft para desarrollar componentes de software distribuidos sobre varias computadoras y que se comunican entre sí.

- **Arquitectura de servicios web.**

Surge a finales de los años 90, tras el poco éxito de CORBA.

Debido a la estandarización del uso de HTTP, desde mediados de la década de los 90, para la de interoperabilidad entre componentes distribuidos y las ventajas del uso de las tecnologías web para los servicios software, se crea un estándar, W3C define una arquitectura de servicios web.

Con la necesidad de integración entre sistemas muy heterogéneos, tanto software como hardware. Muchas empresas comenzaron grandes proyectos para lograr la mejor tecnología integradora de sistemas, percatándose de la imposibilidad de crear una plataforma integrada de forma individual, se buco un lenguaje común de intercambio de información aprovechando los estándares existentes en el mercado, nacen así los Servicios Web basados en XML.

La definición de W3C aporta la información relevante sobre la estructura y patrón de interacción de un servicio web, siendo necesario el rol de un proveedor de servicio, y un consumidor del mismo. Y También debe poderse localizar en la red tal funcionalidad, para lo cual existe el registro del servicio (componente que actúa de directorio de servicios).

Esta arquitectura contribuyo a la construcción de la arquitectura SOA, proporcionando el soporte tecnológico más utilizado para su implementación.

- **Orientada a servicios** (SOA del inglés Service-Oriented Architecture).

Este estilo se apoya en la orientación a servicios, que es una forma de pensar en servicios, su construcción y sus resultados.

- **Arquitectura de microservicios** (MSA del inglés MicroServices Architecture).

Algunos consideran que es una especialización de una forma de implementar SOA.



Consiste en construir una aplicación como un conjunto de pequeños servicios, los cuales se ejecutan en su propio proceso y se comunican con mecanismos ligeros (normalmente una API de recursos HTTP).

Cada servicio se encarga de implementar una funcionalidad completa, es desplegado de forma independiente, puede estar programado en distintos lenguajes y usar diferentes tecnologías de almacenamiento de datos.

- **Dirigida por eventos** (Event-Driven Architecture o EDA).

Es un patrón de arquitectura software que promueve la producción, detección, consumo de, y reacción a eventos. Un evento puede ser definido como "un cambio significativo en un estado".

Lo que es producido, publicado, propagado, detectado o consumido es un mensaje (típicamente asíncrono) llamado notificación del evento, y no el evento en sí mismo, el cual es el cambio de estado que disparó la emisión del evento. Los eventos no viajan, solamente ocurren.

Un sistema dirigido por eventos está compuesto típicamente de:

- Emisores de eventos (o agentes).
- Y consumidores de eventos (o "sink" en inglés).

Los consumidores tienen la responsabilidad de llevar a cabo una reacción tan pronto como el evento esté presente. La reacción puede o no puede ser completamente proporcionada por el consumidor en sí mismo. Por ejemplo, el consumidor debe tener solamente la responsabilidad de filtrar, transformar y reenviar el evento a otro componente o debe proporcionar una reacción propia a algún evento.

Esta arquitectura puede ser aplicada por el diseño e implementación de aplicaciones y sistemas que transmitan eventos entre componentes software que estén emparejados libremente y servicios. También puede complementar la arquitectura orientada a servicios (SOA) porque los servicios pueden ser activados por disparadores que se encuentran en eventos entrantes.

- **Basada en el espacio.**

Esta arquitectura está diseñada específicamente para abordar y resolver problemas de escalabilidad y concurrencia, para evitar el colapso funcional bajo una gran carga al dividir tanto el procesamiento como el almacenamiento entre múltiples dispositivos de un servidor y otro, y también es un patrón útil para las aplicaciones que tienen volúmenes de usuarios concurrentes variables e impredecibles.

La alta escalabilidad se logra eliminando la restricción de la base de datos central y utilizando en su lugar cuadrículas de datos replicados en memoria.

Las arquitecturas basadas en el espacio no suelen estar desacopladas y distribuidas.

Algunas de sus ventajas son:

- Respuesta rápida ante un entorno en constante cambio.
- Son dinámicas, y muy escalables ya que se depende poco o nada de una base de datos centralizada, con lo que se elimina esencialmente este cuello de botella limitante de la ecuación de la escalabilidad en un servidor.



- Con las herramientas basadas en la nube se simplifica su despliegue a los servidores.
- Se logra un alto rendimiento en el servidor gracias al acceso a los datos en memoria y a los mecanismos de almacenamiento en caché incorporados en esta pauta.

Su uso es recomendable para Datos de gran volumen como flujos de clicks y registros de usuarios, datos de bajo valor que pueden perderse ocasionalmente sin grandes consecuencias y redes sociales.

1.2. Interoperabilidad entre sistemas

1.2.1. Definición y objetivos

Proponemos aquí una definición complementaria de "*Interoperabilidad*", a la facilitada en el Bloque I, en la Unidad "Acceso Electrónico de los ciudadanos..." por el Marco Iberoamericano de Interoperabilidad.

La interoperabilidad entre sistemas se refiere a la capacidad de diferentes sistemas de información, plataformas o aplicaciones -incluso habiendo sido desarrollados con tecnologías distintas- para intercambiar datos, interpretarlos correctamente y utilizarlos de forma efectiva. Esta capacidad es esencial en entornos complejos donde coexisten soluciones heterogéneas, como las administraciones públicas, los servicios sanitarios o las empresas con sistemas heredados.

El objetivo principal de la interoperabilidad es garantizar una comunicación fluida y significativa entre sistemas que, de otro modo, estarían aislados. Esto permite que los datos fluyan sin barreras técnicas, semánticas u organizativas, lo cual se traduce en mejoras en la eficiencia operativa, reducción de costes, agilidad en la toma de decisiones y, en el ámbito público, mejor servicio al ciudadano.

Entre los objetivos específicos de la interoperabilidad destacan:

- **Evitar la fragmentación** tecnológica entre departamentos o instituciones.
- **Reutilizar información** ya disponible sin duplicidades.
- **Facilitar la integración** de servicios digitales a través de interfaces estándar.
- **Garantizar la coherencia** y consistencia de los datos compartidos.
- **Permitir la evolución** tecnológica sin romper la compatibilidad con los sistemas existentes.

1.2.2. Niveles de Interoperabilidad

Resumimos aquí un contenido tratado ya en el bloque I, "Acceso electrónico de los ciudadanos...".

La interoperabilidad se estructura habitualmente en distintos niveles, cada uno de los cuales resuelve barreras de diferente naturaleza en el intercambio y uso de la información. Estos niveles no son excluyentes, sino complementarios, y su adecuada coordinación es esencial para lograr una interoperabilidad efectiva y sostenible.



Interoperabilidad técnica

Este nivel se refiere a la capacidad de los sistemas y aplicaciones para **conectarse entre sí y compartir datos mediante infraestructuras, redes, interfaces, lenguajes de intercambio y formatos de datos compatibles**. Incluye el uso de protocolos de comunicación estándar (como HTTP, FTP, SOAP o REST), estructuras de datos comunes (como XML o JSON) y mecanismos de seguridad (como cifrado o autenticación).

Interoperabilidad semántica

Va más allá del intercambio de datos: garantiza que el significado de los datos intercambiados sea entendido por todas las partes implicadas. Para ello, es necesario definir estructuras semánticas comunes (ontologías, vocabularios controlados, taxonomías) que permitan interpretar correctamente la información. Por ejemplo, que "fecha de nacimiento" tenga la misma definición, formato y sentido en todos los sistemas que la usan.

Interoperabilidad organizativa

Afecta a los procesos, políticas y acuerdos entre organizaciones que desean colaborar. Implica establecer mecanismos de coordinación, responsabilidades claras, acuerdos de nivel de servicio (SLA), gobernanza del dato y flujos de trabajo interoperables. Este nivel asegura que la interoperabilidad técnica y semántica se traduzca en un funcionamiento real entre entidades.

Interoperabilidad legal

Se refiere a la adecuación normativa que permite o regula el intercambio de datos entre sistemas, especialmente cuando hay distintos organismos implicados. Esto incluye el cumplimiento del Reglamento General de Protección de Datos (RGPD), la Ley 39/2015 del Procedimiento Administrativo Común, o la Ley 40/2015 de Régimen Jurídico del Sector Público, entre otras.

Estos niveles están reconocidos en normativas internacionales y nacionales como el Esquema Nacional de Interoperabilidad (ENI), que establece los principios básicos y guías para su aplicación en el ámbito de las Administraciones Públicas españolas.

1.2.3. Protocolos y estándares comunes

La interoperabilidad entre sistemas requiere el uso de protocolos y estándares que garanticen el entendimiento mutuo, la compatibilidad tecnológica y la eficiencia en los intercambios. Estos estándares abarcan desde formatos de datos hasta lenguajes de comunicación y modelos semánticos. A continuación se presentan los más utilizados en distintos niveles de interoperabilidad:



Protocolos de comunicación e intercambio de datos

- **HTTP/HTTPS** (Hypertext Transfer Protocol): Protocolo básico de la web, utilizado también para APIs RESTful. HTTPS añade una capa de seguridad mediante cifrado TLS, desarrollados en la Ud "Internet: arquitectura de red..." del bloque IV.
- **FTP/SFTP** (File Transfer Protocol / Secure FTP): Utilizados para la transferencia de archivos entre sistemas, ya sea de forma abierta o segura. Protocolos tratados en la Ud "El modelo TCP/IP..." del bloque IV.
- **SOAP** (Simple Object Access Protocol): Protocolo basado en XML para el intercambio estructurado de información entre servicios web, usado en entornos corporativos por su robustez y estandarización. Hablamos de este protocolo un poco más adelante en esta misma unidad.
- **REST** (Representational State Transfer): Arquitectura para servicios web más ligera que SOAP, basada en HTTP y normalmente utilizando JSON o XML para representar recursos. Tratado en esta misma unidad, un poco más adelante.
- **MQTT** (Message Queuing Telemetry Transport): Es un protocolo ligero y eficiente, diseñado para comunicaciones en entornos con recursos limitados, como el Internet de las Cosas (IoT). Opera sobre TCP/IP y se basa en un modelo de publicación/suscripción con distintos niveles de calidad de servicio, lo que lo hace ideal para sensores, dispositivos móviles o entornos industriales.
- **AMQP** (Advanced Message Queuing Protocol) es un protocolo más robusto, orientado a sistemas empresariales que requieren fiabilidad, seguridad y transacciones. Define no solo el formato de los mensajes, sino también su enrutamiento y almacenamiento en colas, siendo ampliamente utilizado en banca, comercio electrónico y aplicaciones críticas. Ambos protocolos ofrecen soluciones complementarias según el tipo de infraestructura y necesidades del sistema.

Formatos de datos e intercambio

- **XML** (Extensible Markup Language): Estructura jerárquica, extensible, muy usada para representar datos en servicios web y documentos estructurados, tratado más adelante en esta misma unidad así como en la Unidad de "Aplicaciones Web..." de este mismo bloque.
- **JSON** (JavaScript Object Notation): Formato ligero de datos, muy utilizado en APIs modernas por su simplicidad y compatibilidad con JavaScript y múltiples lenguajes, tratado más extensamente en la Unidad "Aplicaciones Web..."
- **CSV** (Comma-Separated Values): Formato sencillo de intercambio de datos tabulares, especialmente útil para integraciones con hojas de cálculo o bases de datos.
- **RDF** (Resource Description Framework): Estándar del W3C para describir recursos y relaciones en la web semántica, facilita la interoperabilidad semántica y el enlace de datos, tratado más extensamente en la Unidad "Accesibilidad, diseño universal..." del bloque III.



Estándares semánticos y de metadatos

- **Dublin Core:** Conjunto de metadatos estándar para describir contenidos digitales, utilizado en bibliotecas, archivos y repositorios digitales.
- **Schema.org:** Esquema colaborativo impulsado por Google, Microsoft, Yahoo y Yandex para estructurar información y facilitar su uso en buscadores y aplicaciones.
- **SKOS** (Simple Knowledge Organization System): Modelo para representar esquemas de clasificación como tesauros, taxonomías o listas de autoridad.

Normas y marcos legales en interoperabilidad pública

- **Esquema Nacional de Interoperabilidad (ENI):** Marco normativo español que regula la interoperabilidad entre administraciones públicas, incluyendo principios técnicos, semánticos, organizativos y jurídicos.
- **Normas Técnicas de Interoperabilidad (NTI):** Desarrollan aspectos específicos del ENI como firma electrónica, documentos, digitalización o protocolos de datos.

Estos protocolos y estándares aseguran que los sistemas heterogéneos puedan colaborar eficazmente, cumpliendo con requisitos de compatibilidad, seguridad, trazabilidad y accesibilidad.

1.2.4. Casos de uso

La interoperabilidad entre sistemas resulta esencial en entornos donde coexisten plataformas tecnológicas diversas que necesitan intercambiar datos, coordinar procesos o integrarse funcionalmente.

Uno de los casos de uso más comunes es el de las **administraciones públicas**, donde organismos distintos (por ejemplo, Seguridad Social, Hacienda y Ayuntamientos) deben compartir información ciudadana de forma ágil, segura y conforme a la normativa. Esta capacidad evita duplicidades, reduce trámites y mejora la eficiencia administrativa.

En el **ámbito sanitario**, la interoperabilidad permite que los sistemas de información hospitalarios compartan historiales clínicos electrónicos, resultados de pruebas o prescripciones entre centros de salud, farmacias y especialistas. Así se favorece la continuidad asistencial, se reducen errores médicos y se optimiza el tratamiento del paciente.

También en el **entorno empresarial**, especialmente en grandes corporaciones o cadenas logísticas, la interoperabilidad posibilita la integración entre sistemas ERP, CRM y plataformas de proveedores, permitiendo sincronización en tiempo real, control de inventario, facturación automática y trazabilidad del proceso productivo.



Beneficios

Los beneficios de la interoperabilidad son múltiples. Desde el punto de vista técnico, **incrementa la eficiencia** operativa al evitar procesos manuales o redundantes. Desde una perspectiva estratégica, **mejora la calidad del servicio**, favorece la toma de decisiones basadas en datos integrados y permite **adaptarse con mayor rapidez a cambios del entorno**. Además, **fortalece la seguridad y el cumplimiento normativo**, al establecer canales estandarizados y controlados para el intercambio de información.

2. Tipos de sistemas

Desde una perspectiva histórica, podemos hablar de diferentes modelos que determinan la funcionalidad y la estructura de un sistema de cómputo, así como las características del sistema operativo como gestor de los recursos y su campo de aplicación y uso.

Vamos a diferenciar 6 tipos de Sistemas:

- **Sistemas por lotes.**

Son los primeros que aparecen. Permiten procesar en diferido y secuencialmente los datos suministrados.

Hoy en día se utilizan en aplicaciones de cálculo intensivo.

- **Sistemas centralizados de tiempo compartido.**

El objetivo es incrementar la eficiencia en el uso de la CPU, disminuyendo los tiempos de respuesta de los usuarios.

Los recursos están centralizados y se accede al sistema desde terminales.

- **Sistemas Personales.**

Sistema dedicado a un único usuario (PC). Su principal característica es un coste reducido. Un sistema personal posee sus propios recursos locales.

Hoy en día, los principales sistemas personales son los PC portátiles y teléfonos móviles.

- **Sistemas de teleproceso.**

Se diferencian del modelo anterior en que los terminales son remotos y acceden a un sistema central utilizando una infraestructura de red y un protocolo de comunicaciones.

El sistema central monopoliza la gestión de los recursos.

- **Sistemas en red.**

Es la evolución del teleproceso, en la que las terminales tienen capacidad de cómputo y se convierten en sistemas autónomos. Desaparece el concepto de ordenador central.



Un sistema en red se compone de un conjunto de ordenadores que se conectan entre sí a través de una infraestructura de red.

La máquina que proporciona el acceso a un recurso determinado (disco duro, impresora, etcétera) se denomina **servidor** del recurso.

Los **clientes** pueden utilizar recursos locales o acceder a recursos remotos (a través de una solicitud al servidor correspondiente).

El desarrollo de protocolos comunes, como TCP/IP, ha permitido interconectar las máquinas independientemente de sus características y sistema operativo, posibilitando el surgimiento de Internet.



Atención

Un tipo de Arquitectura de red se conoce como arquitectura del par-a-par porque cada nodo o caso del programa es un "cliente" y un "servidor" y cada uno tiene responsabilidades equivalentes.

- **Sistemas Distribuidos.**

Vas a estudiar este tipo de sistema con mayor profundidad.

2.1. Tipo: sistemas distribuidos



Fuente:

https://commons.wikimedia.org/wiki/File:Exemplo_de_estrutura_de_sistema_distribu%C3%ADo.jpg

Vamos a estudiar los Sistemas Distribuidos detenidamente.



La tendencia hacia este tipo de sistemas es cada vez mayor. Actualmente tanto el hardware necesario para equipos como para la red de comunicaciones (Ethernet, red telefónica o red eléctrica) tienen un bajo coste.

Ya no existe la diferencia entre recursos locales y remotos. La ubicación del recurso es transparente tanto a las aplicaciones como a los usuarios.

Los sistemas distribuidos proporcionan de forma transparente la compartición de recursos en diferentes máquinas de la red, facilitando el acceso y la gestión, e incrementando la eficiencia y la disponibilidad.



Aviso

Un sistema distribuido es un conjunto de ordenadores interconectados que comparten un estado, ofreciendo una visión de sistema único.

Un sistema en red puede definirse como un conjunto de sistemas con estados independientes, en un sistema distribuido se define un estado global.

El usuario y las aplicaciones no ven una red, sino un sistema indistinguible de uno centralizado. Los protocolos de red se encargan de ocultar la topología y los atributos físicos de la red. La arquitectura de cada máquina la oculta el sistema operativo.

Como los componentes de un sistema distribuido pueden ser heterogéneos, se requiere una capa de software intermedia (middleware) para proporcionar la visión de sistema único.

En computación distribuida, GIOP (Protocolo Entre ORBs General, General Inter-ORB Protocol) es el protocolo abstracto por el cual los ORBs se comunican. Los estándares asociados con el protocolo son mantenidos por el Object Management Group (OMG).

ORB (Object Request Broker) en computación distribuida, es el nombre que recibe una capa de software (llamada middleware) que permite a los objetos realizar llamadas a métodos situados en máquinas remotas, a través de una red. Maneja la transferencia de estructuras de datos, de manera que sean compatibles entre los dos objetos, utilizando un estándar para convertir las estructuras de datos en un flujo de bytes, conservando el orden de los bytes entre distintas arquitecturas, proceso que se denomina marshalling (y también está su opuesto, llamado unmarshalling).

ORB, básicamente permite a objetos distribuidos interactuar entre sí de manera transparente, es decir, como si estuviesen en la misma máquina.

IIOP (Internet Inter-Orb Protocol) es la implementación de GIOP para TCP/IP. Es una realización concreta de las definiciones abstractas de GIOP.



2.1.1. Características de los sistemas distribuidos

- **Concurrencia:** permite que los recursos disponibles en la red puedan ser utilizados simultáneamente por los usuarios que interactúan en la red.
- **Carencia de reloj global:** Las coordinaciones para la transferencia de mensajes entre los diferentes componentes para la realización de una tarea, no tienen una temporización general, está más bien distribuida a los componentes.
- **Fallos independientes de los componentes:** si un componente del sistema falla, los demás pueden continuar ejecutando sus acciones, el sistema en conjunto continúa trabajando, logrando mayor efectividad en las tareas.

2.1.2. Propiedades de los sistemas distribuidos

Un sistema distribuido, debe ofrecer una visión de sistema único, y para ello debe cumplir las siguientes propiedades:

- Transparencia.
- Escalabilidad.
- Fiabilidad y Tolerancia a fallos.
- Consistencia.

Vamos a ver cada uno de estas propiedades con mayor profundidad.

2.1.2.1. Transparencia

La transparencia es el objetivo principal de un sistema distribuido.

Los usuarios y aplicaciones deben percibir los recursos del sistema como si estuvieran gestionados por una sola máquina.

La distribución física de los recursos debe ser transparente.

Hay diferentes tipos de transparencia.

Los 2 tipos de transparencia más importantes son:

- **De identificación:** los espacios de nombres de los recursos son independientes de la topología de la red y de la propia distribución de los recursos.

Una aplicación puede referirse a un recurso con un nombre independientemente de en qué nodo se ejecute.



- **De la ubicación física de los recursos:** ni los usuarios ni las aplicaciones conocen si el recurso es local o remoto y, en caso de ser remoto, no sabemos en qué nodo reside.

Esto implica también que los recursos pueden migrar entre nodos sin afectar a las aplicaciones o usuarios.

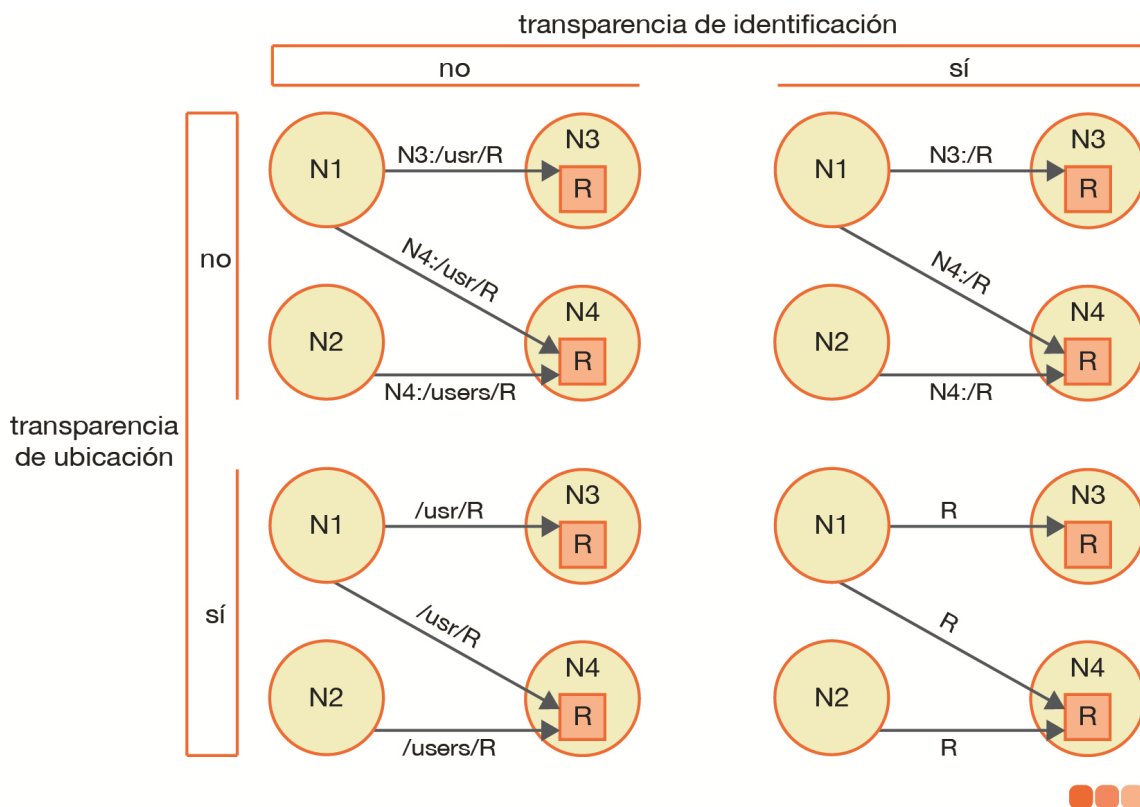


Ejemplo

Ejemplo de estos dos tipos de transparencia:

Podemos montar el sistema de directorios de servicios en la nube, como Google Drive y Dropbox, de manera que lo percibamos como una carpeta más de nuestro sistema de archivos.

Veamos un gráfico para entender mejor estos dos tipos de transparencia y como se integran:



Transparencia de identificación y de ubicación



Otros tipos de transparencia son:

- **De replicación:** ni los usuarios ni las aplicaciones conocen cuántas unidades hay de cada recurso, ni si se añaden o eliminan copias del recurso. La replicación es compleja de gestionar.
- **De paralelismo:** una aplicación puede ejecutarse en paralelo sin que la aplicación tenga que especificarlo y sin consecuencias sobre la ejecución (salvo por cuestiones de rendimiento).
- **De compartición:** el acceso simultáneo a un recurso compartido por parte de varias aplicaciones no debe tener ningún efecto sobre la ejecución.
- **De rendimiento:** implementar las propiedades de los sistemas distribuidos tiene un coste de rendimiento, por lo que hay que buscar un compromiso entre ambos.

2.1.2.2. Escalabilidad

Una de las características de los sistemas distribuidos es su modularidad. Esto les permite una gran flexibilidad y hace que sean altamente escalables.



+ Info

Un sistema es escalable cuando:

Podemos ampliar o reducir sus recursos en función de nuestras necesidades con poco esfuerzo, sin aumentar la complejidad del sistema y sin disminuir su rendimiento.

Uno de los objetivos del diseño de un sistema distribuido es extender la escalabilidad a la integración de servicios.

La escalabilidad presenta dos aspectos:

- **Espacios de nombres:** Proporcionar espacios de nombres suficientemente amplios, de forma que no supongan una limitación al crecimiento del sistema.

Los espacios de nombres pueden identificar distintos tipos de objetos (ficheros, procesos, variables, direcciones de memoria, etcétera).

En el caso de los espacios lineales (como la memoria) existe una limitación inherente asociada al tamaño del nombre, por lo que existe una insuficiencia en el espacio de nombres.

En otros casos, los espacios de nombres son jerárquicos y, por lo tanto, escalables.



- **Complejidad y rendimiento:** Mantener un buen nivel de rendimiento en el acceso a los recursos cuando el sistema crece.

El crecimiento de un sistema distribuido puede introducir cuellos de botella y latencias que degradan su rendimiento. Además, se produce un incremento de los costes de comunicación.

La complejidad de los algoritmos distribuidos no suele ser lineal respecto al tamaño del sistema (aumentan en un rango mayor).

Dos elementos son lineales cuando, al aumentar uno de ellos al doble de su tamaño, el otro también aumenta el doble.

En este caso, al decir que no es lineal, queremos expresar que, si aumentamos el tamaño del sistema al doble, la complejidad de los algoritmos no aumenta el doble sino más (tres o cuatro veces más, por poner un ejemplo).

Por lo tanto, es necesario establecer un compromiso entre tamaño del sistema, rendimiento y complejidad.

2.1.2.3. Fiabilidad y Tolerancia a fallos

La fiabilidad de un sistema es su capacidad para realizar correctamente y en todo momento las funciones para las que se ha diseñado.

La fiabilidad se concreta en dos aspectos:

- **Disponibilidad:** Es la fracción de tiempo que el sistema está operativo. El principal parámetro para medir la disponibilidad es el tiempo medio entre fallos (MTBF), pero habría que considerar también el tiempo que tarda en volver a funcionar tras un fallo.

La disponibilidad se puede incrementar de dos formas:

- Utilizando componentes de mayor calidad.
 - Replicando componentes para que el sistema siga operando, aunque algunos de ellos fallen (aunque con un rendimiento menor).
- **Tolerancia a fallos:** La replicación aumenta la disponibilidad, pero no garantiza por sí sola la continuidad del servicio de forma transparente.

La tolerancia a fallos expresa la capacidad del sistema para seguir operando correctamente ante el fallo de alguno de sus componentes, enmascarando el fallo al usuario o a la aplicación.

Por lo tanto, la tolerancia a fallos implica:

- Detectar el fallo.
- Continuar el servicio de forma transparente para la aplicación (transparencia de fallos).



La probabilidad de fallo disminuye como una función exponencial de la replicación, aunque la ausencia de fallos en un componente nunca puede garantizarse al 100% por muy alta que sea su calidad.

Ejemplo:

Si la probabilidad de que falle un disco duro es de $1/100$ (1%), si duplicamos el disco (aumentamos al doble), la probabilidad de fallo de todos los discos al mismo tiempo no disminuye el doble [$1 / (100 + 100)$] sino exponencialmente [$1 / (100^2) = 1 / 10.000$], ya que para que falle completamente el sistema deben fallar los dos.

Sin embargo, hay que tener en cuenta que la probabilidad de que falle uno de los dos es el doble que la probabilidad de que falle un disco único.

2.1.2.4. Consistencia

El programador, debe garantizar que la memoria será consistente y el resultado de las operaciones de memoria será predecible, para lo que debe seguir unas determinadas reglas.

Los compiladores pueden reordenar instrucciones de memoria, las llamadas a bibliotecas y encapsular la sincronización necesaria para mantener el modelo.

Los clientes deben obtener los mismos resultados al acceder a los mismos datos. El resultado nunca debe ser diferente entre aplicaciones distintas, ni dentro de una misma aplicación.

Es necesario mantener un estado global consistente en un sistema con varios componentes, cada uno de los cuales posee su propio estado local. (A veces esto puede ser un problema).

Los nodos del sistema se hallan físicamente distribuidos, por lo que la gestión del estado global depende fuertemente de los mecanismos de comunicación, a su vez soportados por una red sujeta a fallos.

El mantenimiento de una consistencia estricta requiere un fuerte soporte que implica gran carga de comunicación adicional entre los nodos del sistema.

Muchas veces es preferible relajar la consistencia para mantener el rendimiento en un nivel suficiente (según las necesidades de las aplicaciones).

Por tanto, la distribución de recursos tiene importantes beneficios, pero también ciertos problemas.

- **Beneficios:**

- Contribuye al incremento del rendimiento a través del paralelismo y promoviendo el acceso a copias locales del recurso (disminuyendo los costes de comunicación).
- La replicación aumenta la disponibilidad, siendo la base para proporcionar tolerancia a fallos.



- **Problemas:**

- La red de interconexión es una nueva fuente de fallos.
- La seguridad del sistema es más vulnerable ante accesos no permitidos.
- La gestión para evitar inconsistencias es mucho más compleja.

2.1.3. Aplicaciones distribuidas



Recuerda

Una aplicación paralela es aquella que puede dividirse en tareas que se ejecutan concurrentemente en diferentes elementos de proceso, con el objetivo de disminuir el tiempo de finalización.

Las aplicaciones distribuidas son parecidas, pero presentan motivaciones diversas y se aplican en entornos variados.

Una aplicación distribuida, es una aplicación con distintos componentes que se ejecutan en entornos separados, normalmente en diferentes plataformas conectadas a través de una red. Las típicas aplicaciones distribuidas son:

- De dos niveles (cliente-servidor).
- Tres niveles (cliente-middleware-servidor).
- Multinivel.

2.1.3.1. Objetivos de las Aplicaciones Distribuidas

Sus principales objetivos son:

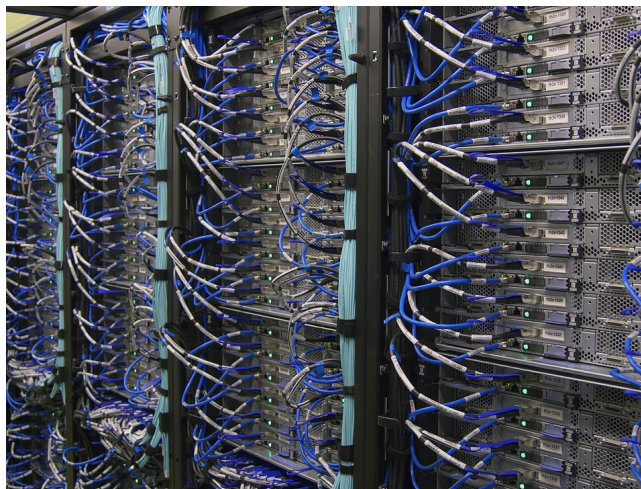
- Alto rendimiento.
- Tolerancia a fallos.
- Alta disponibilidad.
- Movilidad.
- Ubicuidad.



Alto rendimiento

Una aplicación paralela puede ser también distribuida. Por ejemplo, puede utilizarse una red local para distribuir los procesos de la tarea entre los nodos de la red con el fin de aprovechar los recursos de cómputo disponibles (generalmente PC de bajo coste) para reducir el tiempo de finalización.

Precisamente, este tipo de esquema de cómputo (computación en clúster) ofrece hoy una excelente relación rendimiento/coste y se encuentra en expansión frente a los tradicionales supercomputadores.



Computación en clúster

En redes de área amplia se habla de computación en grid.

En este caso, la disponibilidad de recursos para la aplicación es abierta y abarca unidades de cómputo dispersas que en ese momento están ociosas.

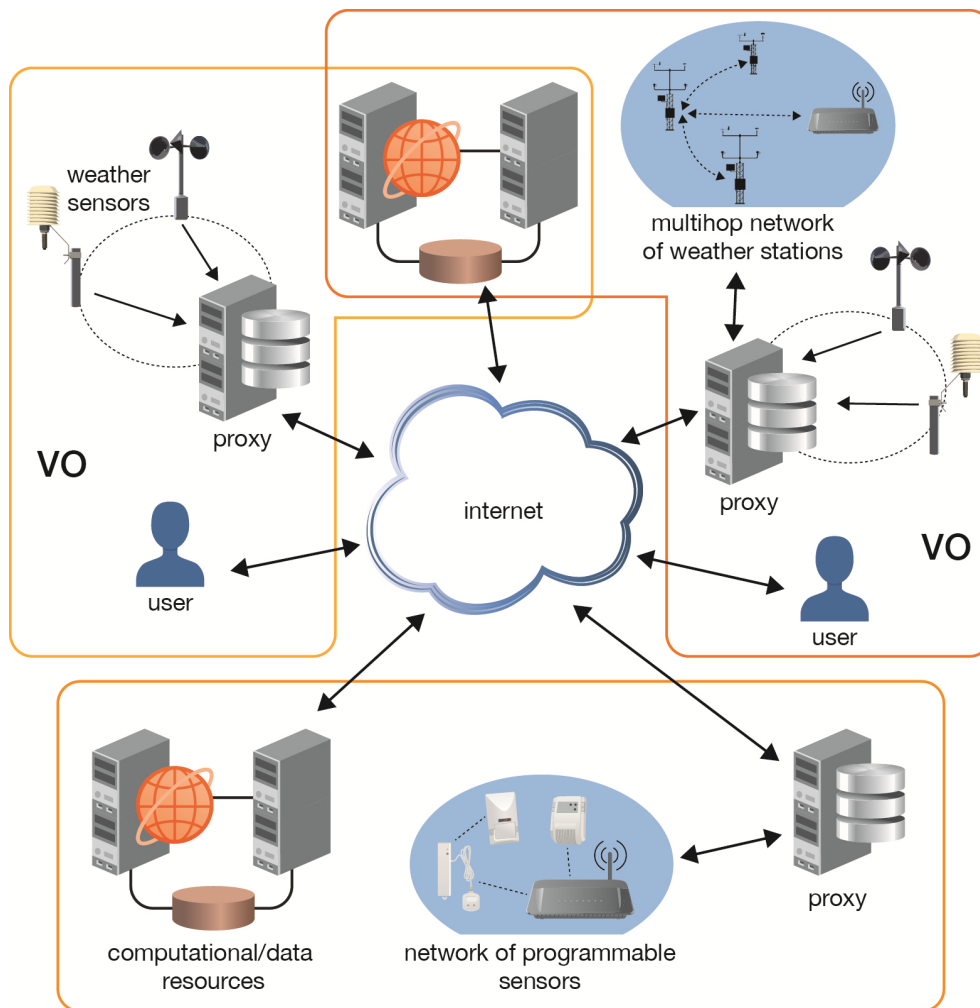
La computación grid (o computación en malla)

Una grid es un conjunto de máquinas distribuidas que ayudan a mejorar el trabajo sobre software pesados.

Sistema de computación distribuido que permite compartir recursos (elementos de hardware, software, datos e información o personas) no centrados geográficamente para resolver problemas de gran escala.

Ofrece muchas ventajas, la potencia que ofrecen multitud de computadores conectados en red usando grid es prácticamente ilimitada, además de que ofrece una perfecta integración de sistemas y dispositivos heterogéneos, por lo que las conexiones entre diferentes máquinas no generarán ningún problema.

Se trata de una solución altamente escalable, potente y flexible, ya que evitarán problemas de falta de recursos (cuellos de botella) y nunca queda obsoleta, debido a la posibilidad de modificar el número y características de sus componentes.



Arquitectura grid de sensores (fuente: https://en.wikipedia.org/wiki/File:Sensor_Grid_architecture-new.jpg)

Tolerancia a fallos

Hoy en día se utilizan técnicas muy conservadoras que sacrifican transparencia para mejorar la tolerancia a fallos.



Ejemplo

En aplicaciones críticas (control de central nuclear o de máquinas de soporte vital) el fallo de una máquina resulta inaceptable.



Alta disponibilidad

Hay aplicaciones donde la distribución se realiza para acercar la información al usuario y disminuir los tiempos de respuesta. En estos casos se busca la escalabilidad.

Los sistemas *peer-to-peer* (P2P) poseen una gran escalabilidad al evitar los cuellos de botella del servidor.

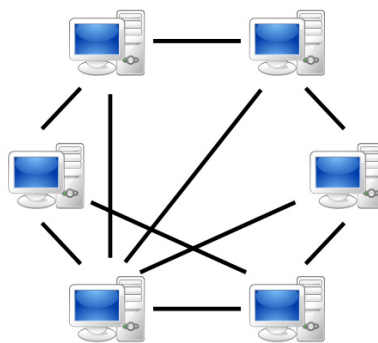
Un sistema *peer-to-peer* es una red de ordenadores en la que los nodos se comportan como iguales entre sí. Es decir, actúan simultáneamente como clientes y servidores respecto a los demás nodos de la red.

Las redes P2P permiten el intercambio directo de información, en cualquier formato, entre los ordenadores interconectados.



+ Info

Seguro que has usado este tipo de redes alguna vez. BitTorrent, Edonkey o Emule son ejemplos de aplicaciones P2P.



Red P2P (fuente:
<https://commons.wikimedia.org/wiki/File:P2P-network.svg>)

Movilidad

En la actualidad utilizamos gran cantidad de dispositivos (PC, portátiles, tabletas, teléfonos móviles, etcétera).

Esto dificulta el acceso a nuestra información, ya que al modificar algo en un dispositivo debemos actualizarlo en el resto.



Por lo tanto, es necesario desligar la información del dispositivo y gestionar convenientemente las actualizaciones.

Hoy en día, hay una tendencia a trabajar en espacios virtuales, en lugar de sobre dispositivos concretos. De esta forma, la información está en un solo sitio y puede ser accedida y modificada desde cualquier dispositivo. Sin olvidarnos nunca de la importancia de crear nuestras propias copias de seguridad.



Ejemplo

Ejemplos de estos espacios virtuales en la nube son:

- Gmail para el correo electrónico.
- Google Drive, Dropbox y OneDrive para almacenamiento secundario.



Fuente:

<https://www.flickr.com/photos/111692634@N04/16203260320>

Ubicuidad

A veces los recursos están inherentemente distribuidos. El usuario se mueve en un entorno con recursos ubicuos (que están presentes en todas partes) y la aplicación trata de ofrecer un comportamiento inteligente en función de las necesidades del usuario y la naturaleza y disponibilidad de los recursos.

La visión principal de la inteligencia ambiental presenta al usuario rodeado de interfaces inteligentes e intuitivas, integradas en los objetos cotidianos de su entorno de forma transparente.



Estas interfaces poseen capacidad para reconocer la presencia de diferentes usuarios. Modifican su comportamiento en función de la identidad de dicho usuario, sus necesidades y las características del contexto o entorno donde se encuentren.



Ejemplo

Las aplicaciones de **inteligencia ambiental** están basadas en este principio de Ubicuidad.

Un sistema de audio podría identificar a una persona y su estado de ánimo y poner música en consecuencia.



Fuente: <https://pixabay.com/es/artificial-inteligencia-robot-bordo-2970158/>

3. Arquitectura cliente/servidor

Cliente-Servidor es un modelo de diseño de software, las tareas se reparten entre los proveedores de recursos o servicios, llamados servidores, y los demandantes de estos servicios a los que se les llama clientes.

Un cliente realiza peticiones al servidor, quien le da respuesta.

Proporciona grandes ventajas en un sistema multiusuario distribuido a través de una red de computadoras. (también se puede aplicar a programas que se ejecutan sobre una sola computadora).



Ejemplo

Aplicaciones modelo cliente-servidor son:

Correo electrónico, juegos en red, un Servidor de impresión y la World Wide Web.

En esta arquitectura se reparte la capacidad de proceso entre los clientes y los servidores.

Es importante la ventaja de tipo organizativo que se obtiene centralizando la gestión de la información y realizando separación de responsabilidades, lo que hace que el diseño del sistema sea más fácil y claro.

La separación entre cliente y servidor es una separación de tipo lógico; el servidor no tiene por qué ser una sola máquina ni un solo programa. (También puede darse la acción cliente-servidor en una misma máquina).

En todas las gestiones que se realizan en el servidor, es éste el que determina se disponen los requerimientos provenientes de los clientes que tienen prioridad, los archivos que son de uso público y los que son de uso restringido, los archivos que son de sólo lectura y los que, por el contrario, pueden ser modificados, etc.

La arquitectura básica será siempre la misma, aunque varíen los tipos de servidores según sus propósitos (servidores específicos) cómo los servidores web, los servidores de archivo, los servidores del correo, etc.

Son muy comunes los sistemas multicapa en los que el servidor se descompone en diferentes programas que pueden ser ejecutados por diferentes computadoras aumentando así el grado de distribución del sistema.

La estructura cliente/servidor es una arquitectura en la que uno o varios clientes, que pueden estar distribuidos geográficamente, solicitan servicios a uno o más servidores (los cuales también pueden estar distribuidos geográficamente).

La arquitectura cliente/servidor es una arquitectura distribuida. Las tareas se reparten entre distintos servidores de forma transparente al usuario.

3.1. Características

Un sistema cliente/servidor está basado en las siguientes características:

- **Es un sistema distribuido.**
- **Independencia de la plataforma:** *en software y en hardware.*

El **ambiente es heterogéneo**. La plataforma de hardware y el sistema operativo del cliente y del servidor no son siempre los mismos, se conectan clientes y servidores independientemente de sus plataformas.



- **Interoperabilidad de plataformas:** Las funciones de Cliente y Servidor pueden estar en plataformas separadas, o en la misma plataforma.
- **Recursos compartidos:** se comparten recursos tanto lógicos como físicos.
- **Servicio:** es la unidad básica de diseño. El servidor los ofrece y el cliente los solicita y usa.
- **Encapsulamiento de servicios:** los detalles de la implementación de un servicio son transparentes al cliente.
- **Combinación** de un cliente que interactúa con el usuario, y un servidor que interactúa con los recursos a compartir.
 - El proceso del cliente proporciona la interfaz entre el usuario y el resto del sistema.
 - El proceso del servidor actúa como un motor de software que maneja recursos compartidos tales como bases de datos, impresoras, Módem, etc.
- Las **tareas** del cliente y del servidor tienen diferentes requerimientos en cuanto a recursos de cómputo como:
 - Velocidad del procesador.
 - Memoria.
 - Velocidad y capacidades del disco.
- **Protocolos asimétricos:** los clientes inician la conversación. Los servidores están escuchando. Atenderán a los clientes cuando puedan según el número de peticiones de clientes.
- **Sistemas débilmente acoplados:** interacción basada en envío de mensajes.
- **Relación cliente-servidor:** La única relación es la que se establece a través del intercambio de mensajes entre ambos. El mensaje es el mecanismo para la petición y entrega de solicitudes de servicios. No existe otra relación.

Puede ser:

- Entre procesos distintos, que pueden ser ejecutados en la misma máquina o en máquinas diferentes distribuidas a lo largo de la red.
- De muchos a uno: un servidor puede dar servicio a muchos clientes, regulando su acceso a los recursos compartidos.
- Existe una clara **distinción de funciones**, cuando un cliente realiza una petición a un servidor, atendiendo a quien procesa el trabajo, los clientes y servidores pueden ser activos o pasivos:
 - Activos. Realizan el trabajo indicado.
 - Pasivos. Sólo procesan información.

El Cliente y el Servidor pueden actuar como una sola entidad y también pueden actuar como entidades separadas, realizando actividades o tareas independientes.



- **Escalabilidad:** horizontal y vertical, es aplicable a cualquier sistema Cliente-Servidor.
 - **Escalabilidad horizontal:** aumento de la capacidad del sistema agregando servidores físicos o máquinas virtuales, contenedores, nodos de procesamiento o de almacenamiento...
 - **Escalabilidad vertical:** mejora de las características del servidor, reemplazando los procesadores instalados por otros más potentes, aumentando la cantidad memoria RAM, o la tecnología de almacenamiento.

También, la escalabilidad vertical posibilita el aumento de la capacidad de cálculo de los servidores.

Cada plataforma puede ser **escalable independientemente**. Los cambios realizados en las plataformas de los Clientes o de los Servidores, ya sean por actualización o por reemplazo tecnológico, se realizan de una manera **transparente** para el usuario final.

- **Transparencia de localización física de los servidores y clientes.** El cliente no tiene por qué saber dónde se encuentra situado el recurso que desea utilizar.

La interrelación entre el hardware y el software es una gran infraestructura, pero el acceso a los recursos de la red no muestra la complejidad de los diferentes tipos de formatos de datos y de los protocolos.

- **Integridad:** tener los datos y aplicaciones centralizados en servidores facilita su integridad y mantenimiento.

3.2. Tipos de comunicación

Las peticiones que realizan los clientes a los servidores pueden iniciarse en modo síncrono o asíncrono:

- **Comunicación síncrona:** El cliente solicita la operación al servidor y se queda bloqueado hasta que ocurre alguno de los llamados grados de sincronía, que son:
 - Que el nodo remoto recibe el mensaje:
Síncrono con respecto a transmisión (RPC/RMI asíncrona).
 - Que el proceso remoto recibe el mensaje:
Síncrono con respecto a recepción.
 - Que el proceso remoto procese y responda al mensaje:
Síncrono con respecto a respuesta (p.e. RPC/RMI).
(Protocolo petición/respuesta característica de cliente-servidor).



- **Comunicación asíncrona:** el cliente solicita la operación y se le devuelve el control inmediatamente, la solicitud no es bloqueante. La solicitud se realiza mediante mensajes al servidor (o eventos).



+ Info

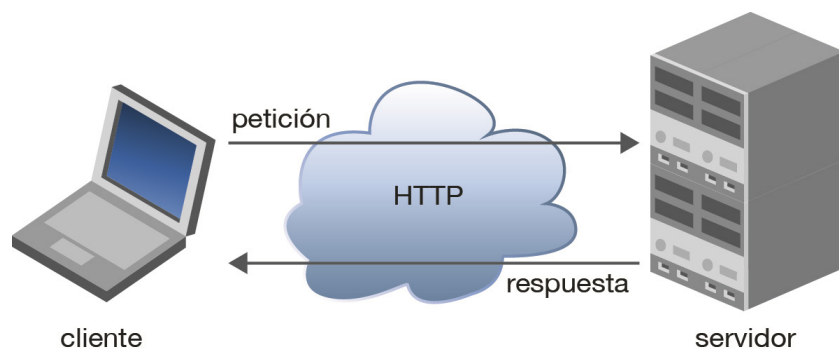
Terminología:

- Accept(): petición de conexión de un cliente).
- Recv(): recepción llamadas de datos.

3.3. Funcionamiento

El esquema de funcionamiento de un sistema cliente/servidor es el siguiente:

- El cliente solicita una información o un servicio al servidor.
- El servidor (que está escuchando) recibe la petición del cliente.
- El servidor procesa dicha solicitud.
- El servidor envía el resultado obtenido al cliente.
- El cliente recibe el resultado.





Atención

La interacción entre el cliente y el servidor se describe a menudo usando diagramas de secuencia. Los diagramas de secuencia se estandarizan en el UML. Es importante que los clientes no interactúen entre sí ni que lo hagan clientes de capas bajas hacia otros de capas más altas, por eso todo tiene que pasar por el servidor.

En el modelo cliente/servidor, el cliente es un proceso consumidor de servicios y el servidor es un proceso proveedor de servicios.

Esta relación se establece mediante el intercambio de mensajes.

Normalmente, para entender mejor los elementos involucrados a la hora de diseñar y desarrollar aplicaciones en una arquitectura cliente-servidor se utiliza un modelo de descomposición en niveles o capas en función de los aspectos funcionales.

A continuación, proponemos la siguiente descomposición en capas:

- **Nivel de presentación:** agrupa a todos los elementos asociados al componente cliente.
- **Nivel de aplicación:** agrupa a todos los elementos asociados al componente servidor.
- **Nivel de comunicación:** agrupa a todos los elementos que hacen posible la comunicación entre los componentes cliente y servidor.
- **Nivel de base de datos:** agrupa a todas las actividades asociadas al acceso de los datos.

3.4. Componentes

A partir del funcionamiento del modelo cliente servidor, vemos que hay 3 componentes básicos:

- **El cliente.** Es quien inicia el diálogo.
- **El servidor.** Espera a que le lleguen peticiones de servicio.
- **El middleware.** Es la interfaz que provee la conectividad entre cliente y servidor para que puedan intercambiar mensajes.

Vamos a ver cada uno de los componentes con mayor profundidad.



3.4.1. Cliente

Un cliente es todo proceso que reclama servicios de otro.



Atención

Los procesos cliente también se conocen como front-end.

El cliente incluye, el sistema operativo, el interfaz gráfico de usuario (GUI) o interfaz orientado a objetos de usuario (OOUI).

Las funciones que lleva a cabo el proceso cliente son:

- Es quien inicia solicitudes o peticiones, tienen por tanto un papel activo en la comunicación (dispositivo maestro o amo).
- Interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario. (Administrar la interfaz de usuario) Formatear resultados para presentarlos al usuario.
- Procesar la lógica de la aplicación y hacer validaciones locales.
- Generar requerimientos de bases de datos.
- Espera y recibe las respuestas del servidor.
- Generalmente, puede conectarse a varios servidores a la vez.

3.4.2. Servidor

Un servidor es todo proceso que proporciona un servicio a clientes (que pueden ser a su vez servidores). Ejecuta software especializado.

Es el proceso encargado de atender a múltiples clientes que hacen peticiones de algún recurso que administra.



Atención

El proceso servidor también se conoce como back-end.

Por norma general, el servidor maneja las reglas de negocio y los recursos de datos.

Las 4 principales funciones que lleva a cabo el proceso servidor son:

- Al iniciarse espera a que lleguen las solicitudes de los clientes, desempeñan entonces un papel pasivo en la comunicación (dispositivo esclavo).
- Acepta las peticiones sobre datos que realizan los clientes.
- Tras la recepción de una solicitud, procesa la petición y accede a las bases de datos según los requisitos especificados en la petición del cliente.
- Da a los datos el formato adecuado para transmitirlos a los clientes y envía la respuesta al cliente.

Por norma general, el servidor se encarga de procesar la lógica de la aplicación.

Generalmente, un servidor acepta las conexiones de un gran número de clientes (en ciertos casos el número máximo de peticiones puede estar limitado).

Los servidores también pueden actuar como clientes de otros servidores.

El término servidor también se utiliza para designar a un ordenador de altas prestaciones que puede dar servicios a un gran número de usuarios de forma concurrente.

Desde el punto de vista de la arquitectura cliente/servidor, un servidor es un servicio software que atiende peticiones de procesos clientes.



+ Info

Los servidores pueden ser apátridas o stateful.



Un servidor apátrida no guarda ninguna información entre las peticiones.

- Un servidor stateful puede recordar la información entre las peticiones.
- El alcance de esta información puede ser global o sesión-específico.

Un servidor del HTTP para las páginas estáticas del HTML es un ejemplo de un servidor, apátrida mientras que Apache Tomcat es un ejemplo de un servidor stateful.

3.4.3. Middleware

El *middleware* es un módulo intermedio que actúa como conductor entre sistemas, permitiendo a cualquier usuario de sistemas de información comunicarse con varias fuentes de información que se encuentran conectadas por una red.

Es un software distribuido para interacciones entre cliente y servidor, y se ejecuta en ambas partes.

Es responsable del buen funcionamiento, especialmente en N niveles.

El middleware, actúa desde la API del cliente usada para invocar el servicio, la transmisión de la solicitud y la respuesta hasta el sistema que informa al servidor.

El *middleware* se estructura en tres niveles:

- Protocolo de transporte (como TCP/IP, IPX...).
- Network Operating System (NOS o sistema operativo de red). (como RPC, Samba...).
- Protocolo específico del servicio. (Middleware específico para el servicio como HTTP, ORB...).

Objetivos del middleware

Utilizando *middleware* para desarrollar arquitecturas cliente/servidor conseguimos:

- Independencia entre servidores y clientes.
- Facilitar la interrelación entre cliente y servidor.
- Evitar dependencias de tecnologías propietarias.



Características del middleware

Vamos a ver las características principales del middleware:

- Simplifica el proceso de desarrollo de aplicaciones al independizar los entornos propietarios.
- Permite la interconectividad de los sistemas de información.
- Proporciona mayor control del negocio.
- Facilita el desarrollo de sistemas complejos con diferentes tecnologías y arquitecturas.



+ Info

Hay que tener en cuenta que no todo son ventajas al utilizar *middleware*.

Necesita recursos para funcionar, por lo que aumenta la carga de procesamiento.

3.5. Tipos de arquitecturas cliente/servidor

Dependiendo de las relaciones entre cliente, servidor y middleware, podemos clasificar en 2 tipos de arquitecturas cliente/servidor:

- Basado en el tamaño de los componentes.
- Basado en la naturaleza del servicio que ofrecen.

Para ver estas relaciones tenemos en cuenta conceptos como:

- La oportunidad de la información.
- El tiempo de respuesta.
- El tamaño de los registros.
- El tamaño de las bases de datos.
- Estimaciones del tráfico de red.
- Distribución geográfica de los distintos elementos.
- Etcétera.



3.5.1. Por el tamaño de los componentes

Este tipo de clasificación se basa en los grados de libertad que brinda el modelo cliente/servidor para balancear la carga de proceso entre los niveles de presentación, aplicación y base de datos.

Dependiendo de qué segmento de las capas de software tenga que soportar la mayor o menor carga de procesamiento, se habla de Fat Client (Thin Server) o Fat Server (Thin Client).

3.5.1.1. Fat Client (Thin Server)

En este esquema el nivel de presentación y el nivel de aplicación corren en el cliente. El servidor tan solo realiza funciones de administrador de base de datos.

Es decir, los niveles de presentación y aplicación están en el cliente y el nivel de base de datos en el servidor.

Tiene pocas posibilidades de aplicarse en sistemas de misión crítica.



Ejemplo

En general, este tipo de arquitectura se suele utilizar para sistemas de apoyo a la decisión (DSS o *Decision Support System*) y sistemas de información ejecutiva (EIS o *Executive Information System*).

3.5.1.2. Fat Server (Thin Client)

Es el caso contrario al anterior. El proceso cliente tan solo funciona como un interfaz de usuario para presentar datos. El servidor se encarga del peso de la aplicación y del acceso a las bases de datos.

Es decir, el nivel de presentación está en el cliente y los niveles de aplicación y de base de datos están en el servidor.

Este tipo de arquitectura ofrece mayor flexibilidad para el desarrollo de aplicaciones, especialmente se utiliza en sistemas de misión crítica (a través de servidores de transacciones).



Atención

Se entienden como Sistemas de Misión Crítica aquellos que son indispensables para que funciones de importancia relevante se lleven a cabo con éxito, ya sea en una empresa, un gobierno o cualquier tipo de organización.

Los Datacenters que soportan las operaciones del sistema financiero, de los sistemas de salud, de la red de seguridad y atención a emergencias de un país o región (ej.*911) y otros similares son ejemplo de Sistemas de Misión Crítica, pero también la PBX de una empresa de CallCenter comercial será vista como un sistema crítico en el análisis de riesgo del negocio.

3.5.2. Por la naturaleza del servicio proporcionado

Podemos diferenciar diferentes servidores, dependiendo del servicio que ofrecen.

- De ficheros.
- De bases de datos.
- De transacciones.
- De objetos.
- Web.

Vamos a estudiar cada uno de ellos.

3.5.2.1. Servidores de ficheros

Los servidores de archivos usan recursos compartidos sobre la red y son necesarios para crear repositorios de documentos, imágenes y archivos grandes. Necesita intercambiar gran cantidad de mensajes sobre la red.



3.5.2.2. Servidores de bases de datos

La creación de aplicaciones cliente/servidor suele estar asociada a la utilización de servidores de bases de datos relacionales SQL. Dependiendo de los requisitos y restricciones se debe elegir entre una arquitectura dos o tres niveles.

Para una arquitectura centrada en un servidor de bases de datos, cualquiera de las modalidades de dos niveles permite que un proceso cliente solicite datos y servicios directamente a un servidor de bases de datos.

El servidor debe proveer un acceso compartido a los datos con mecanismos para:

- La protección de datos.
- La selección del conjunto de datos solicitado.
- De concurrencia.
- De seguridad.
- De consistencia de datos basado en el concepto de transacción.



+ Info

Recordamos que una transacción hace que un grupo de acciones se realicen en su totalidad o que, si falla, no se anule y no se realice nada en absoluto.

De esta manera evitamos que determinadas acciones se realicen a medias, creando un problema de consistencia de datos.

Los servidores de bases de datos soportan SQL y añaden algunas extensiones propias de cada proveedor. La mayoría de las bases de datos están provistas de:

- Procedimientos almacenados: son funciones que agrupan un conjunto de instrucciones y lógica de procedimientos SQL. Se compilan y almacenan en la propia base de datos.

Su función principal es proveer a la parte servidora de la lógica de aplicación (es decir, reemplaza el nivel de aplicación de una arquitectura de tres niveles). Por lo tanto, posibilita implementar el sistema cliente/servidor en dos niveles.



- Desencadenantes (*triggers*): son mecanismos que permiten realizar acciones automáticamente sobre los datos cuando se produce un determinado evento. Normalmente se implementan con procedimientos almacenados.
- Restricciones (*constraints*): también son acciones desencadenadas por un evento, pero están orientadas a validar datos.

3.5.2.3. Servidores de transacciones

Se pueden implementar con cualquier modalidad cliente/servidor (de dos o tres niveles). Se basan en el concepto de transacción.

Con un servidor de transacciones, el proceso cliente llama a funciones, procedimientos o métodos que residen en el servidor, ya se trate de un servidor de bases de datos o un servidor de aplicaciones.

Lo importante es que el intercambio a través de la red se realiza mediante un único mensaje de solicitud/respuesta. No importa el número de funciones, instrucciones o sentencias SQL que hay que ejecutar, todas estarán agrupadas en una unidad lógica (transacción).



Ejemplo

Un ejemplo típico es el cajero de un banco. Si enviamos una solicitud para sacar dinero y el proceso se corta después de descontarlo, habremos perdido el dinero.

Por ello, todos los pasos se deben hacer como un paso único. Si falla cualquier paso, no se realiza ninguna de las acciones anteriores.

3.5.2.4. Servidores de objetos

En un servidor de objetos, las aplicaciones cliente/servidor se escriben como un conjunto de objetos que se comunican. Los objetos cliente se comunican con los objetos servidores usando un *Object Request Broker* (ORB).

El cliente invoca un método de un objeto remoto. El ORB localiza el método del objeto en el servidor y lo ejecuta para devolver el resultado al objeto cliente.

Los servidores de objetos deben soportar concurrencia.



3.5.2.5. Servidores web

La aplicación cliente servidor más importante es la World Wide Web. Este nuevo modelo consiste en clientes simples que realizan solicitudes a servidores web.

Un servidor web devuelve documentos cuando el cliente pregunta por el nombre de estos. Los clientes y los servidores se comunican usando el protocolo HTTP. Este protocolo define un conjunto simple de comandos, los parámetros son pasados como cadenas y no provee tipos de datos.

3.5.3. Patrón de Diseño MVC

MVC, siglas de Modelo-Vista-Controlador.

Es un patrón de arquitectura de software, que separa los datos y la lógica de negocio de una aplicación de su representación y el módulo encargado de gestionar los eventos y las comunicaciones.

MVC define componentes para la representación de la información, y por otro lado para la interacción del usuario, proponiendo la construcción de tres componentes distintos:

- **El modelo.**

Es la representación de la información con la cual el sistema opera, por lo tanto, gestiona todos los accesos a dicha información, implementando también los privilegios de acceso que se hayan descrito en las especificaciones de la aplicación (lógica de negocio).

Envía a la 'vista' aquella parte de la información que en cada momento se le solicita para que sea mostrada al usuario.

Las peticiones de acceso o manipulación de información llegan al 'modelo' a través del 'controlador'.

- **El controlador.**

Responde a eventos recibidos del usuario, e invoca peticiones al 'modelo' cuando se hace alguna solicitud sobre la información.

Hace de intermediario entre la 'vista' y el 'modelo' (Middleware).

También puede enviar comandos a su 'vista' asociada si se solicita un cambio en la forma en que se presenta el 'modelo' (como desplazamiento o scroll por un documento o por los diferentes registros de una base de datos).

- **La vista.**

Presenta el 'modelo' (información y lógica de negocio) en un formato adecuado para la interfaz de usuario) por tanto requiere de dicho 'modelo' la información que debe representar como salida.



Este patrón de arquitectura de software se basa en las ideas de reutilización de código y la separación de conceptos, para facilitar la tarea de desarrollo de aplicaciones y su posterior mantenimiento. Para ello:

- Separa la lógica de negocio de la interfaz de usuario.
- Incrementa la reutilización y la flexibilidad.
- Se ha utilizado en múltiples frameworks (J2EE, ASP, .NET, MVC, ETC.).



+ Info

Aunque originalmente MVC fue desarrollado para aplicaciones de escritorio, ha sido adaptado como arquitectura para diseñar e implementar aplicaciones web. Se han desarrollado multitud de frameworks, comerciales y no comerciales, que implementan este patrón.

Flujo de comunicación en MVC

El flujo de comunicación en **MVC** sigue una secuencia bien definida que comienza cuando el usuario interactúa con la vista. Esta envía la acción al controlador, quien se encarga de procesar la petición y trabajar con el modelo si es necesario. El modelo actualiza los datos y notifica los cambios, momento en el que el controlador selecciona la vista apropiada para mostrar la respuesta al usuario. Es fundamental comprender que la vista nunca se comunica directamente con el modelo, manteniendo así una separación clara de responsabilidades.

Responsabilidades específicas de cada componente

El modelo no solo gestiona los datos y la lógica de negocio, sino que **también se encarga de notificar los cambios a través del patrón Observer**. Es importante destacar que el modelo trabaja de forma independiente, sin conocer ni a la vista ni al controlador. Por su parte, la vista se dedica exclusivamente a la presentación de los datos, limitándose a mostrar la información que recibe sin procesarla. El controlador actúa como el coordinador principal, validando los datos antes de enviarlos al modelo y tomando la decisión sobre qué vista debe mostrarse en cada momento.

Ventajas prácticas del patrón

Entre las ventajas más significativas destaca la posibilidad de desarrollo paralelo, donde **diferentes equipos pueden trabajar simultáneamente en los distintos componentes**. Esta separación facilita enormemente las pruebas individualizadas de cada módulo y simplifica el mantenimiento al permitir localizar errores de manera más rápida y eficiente. La reutilización de componentes en otras partes de la aplicación se ve notablemente mejorada gracias a esta arquitectura.



Consideraciones prácticas

Es importante tener en cuenta que MVC **introduce una complejidad inicial mayor que otros enfoques**, por lo que puede no ser la opción más adecuada para aplicaciones muy simples o prototipos rápidos. La curva de aprendizaje también es un factor a considerar, especialmente para desarrolladores que se enfrentan por primera vez a este patrón de arquitectura.

Ejemplos en tecnologías actuales

Este patrón ha demostrado su vigencia a lo largo del tiempo y se mantiene ampliamente utilizado en frameworks modernos. Encontramos implementaciones en:

- **Spring MVC** para Java.
- **Laravel** para PHP.
- **ASP.NET MVC** en el ecosistema Microsoft.
- **Angular** en el desarrollo frontend.

Cada uno de estos frameworks adapta los principios básicos de MVC a las particularidades de su tecnología, demostrando **la flexibilidad y utilidad de este patrón de diseño en el desarrollo software contemporáneo**.

3.6. Modelos cliente/servidor

Una de las clasificaciones mejor conocidas de las arquitecturas cliente/servidor se basa en el concepto de niveles (*tier*).

Estos definen el modo en que las funcionalidades de la aplicación serán asignadas y en qué proporción, tanto al cliente como al servidor. Dichas prestaciones se deben agrupar en tres partes básicas.

- Interfaz de usuario.
- Lógica de negocios.
- Datos.

Dentro de esta categoría tenemos tres tipos de aplicaciones cliente/servidor:

- De dos niveles (*two-tier*).
- De tres niveles (*three-tier*).
- Multinivel (*multi-tier*).

El concepto de **Nivel** físico hace referencia a la organización y división a nivel físico. Una aplicación de tres capas (o planos) por ejemplo no tiene por qué tener tres niveles.



Confusión entre capas y niveles

Actualmente es muy usado el diseño arquitectónico de sistemas software en capas, y también los despliegues físicos en distintas unidades o servidores.

Es muy frecuente confundir 2 conceptos referentes al diseño de software, y hay que tener muy claro que son conceptos diferentes. Son:

- División en capas: **capa lógica (Layer)**.

Se trata del diseño de la organización y división del código del sistema a un nivel lógico.

No se hace referencia a la ubicación física del despliegue, sino a la organización lógica del sistema.

- División en niveles: **Nivel físico (Tier)**.

Tratamos la organización y división a nivel físico, de los distintos componentes o elementos de diseño que conforman el sistema. Sólo los niveles físicos (Tier) implican una separación física de los componentes desplegados.

Ya sabemos que una de las técnicas más comunes es la división en distintas capas y niveles de abstracción, "Divide y Vencerás", permitiéndonos aislar la forma de desarrollar, si se realizan cambios, solo se verá afectada esa capa en la que se trabaja, facilitando el mantenimiento de los sistemas y reduciendo su coste.

Ejemplo:

En telecomunicaciones, tenemos la pila OSI como división en niveles de abstracción de los protocolos de red. Pero no es lo mismo el concepto de diseño en capas lógicas, que el despliegue en distintos niveles físicos.



Aviso

Nunca hay que confundir Capa con Nivel.

No pienses en que hay tantos niveles físicos como capas lógicas tenga el diseño.

Ejemplo: una aplicación de tres capas no tiene por qué tener tres niveles.



3.6.1. A nivel de Hardware

Este enfoque es menos importante que el orientado al software. Se basa igualmente en la distribución de los procesos y elementos entre sus componentes, pero centrándose en la parte física del mismo.

La interfaz gráfica se asocia con los PC cliente.

La seguridad e integridad de los datos se asocia a los equipos servidores.



+ Info

Si tenemos una complejidad en la capa de negocio lo que obligase a la separación, esta capa de negocio podría residir en uno o más ordenadores que realizarían solicitudes a una única base de datos.

En sistemas muy complejos se llega a tener una serie de ordenadores sobre los cuales corre la capa de negocio, y otra serie de ordenadores sobre los cuales corre la base de datos.

Modelo de un nivel, monolítico

En este tipo el interfaz de usuario, la lógica de negocio y el acceso a las bases de datos estaba todo contenido en una gran aplicación que se ejecutaba en el mainframe (un solo ordenador potente). Dado que las terminales utilizadas para conectarse al ordenador central no tenían ninguna capacidad de proceso, (terminales tontas) la aplicación entera se ejecutaba completamente en el ordenador central.



Ejemplo

Típico ordenador único de gestión que tiene el interfaz de usuario, la aplicación y los datos en un solo entorno físico, pero en tres capas de proceso:

- Interfaz.
- Aplicación.
- Datos.



Modelo de dos niveles (No capas)

Los clientes son conectados vía LAN a un servidor de aplicaciones local, el cual, dependiendo de la aplicación, puede dar acceso a los datos administrados por él.

Modelo de tres niveles (No capas)

Los clientes son conectados vía LAN a un servidor de aplicaciones local, el cual a su vez se comunica con un servidor central de bases de datos.

El servidor local tiene un comportamiento dual, dado que actúa como cliente del servidor de datos y como servidor de los clientes.

3.6.2. A nivel de Software

La programación por capas es un modelo de desarrollo software en el que el objetivo primordial es la separación (desacoplamiento) de las partes que componen un sistema software o también una arquitectura cliente-servidor:

- **Lógica de negocios:**

Es la capa de negocio, donde residen los programas que se ejecutan, se reciben las peticiones del usuario y se envían las respuestas tras el proceso. Esta capa establece todas las reglas que deben cumplirse, y se comunica con la capa de presentación, para recibir las solicitudes y presentar los resultados, y con la capa de datos, para solicitar al gestor de base de datos almacenar o recuperar datos de él. (También se consideran aquí los programas de aplicación).

- **Capa de presentación:**

Es la que ve el usuario (también llamada capa de usuario), presenta el sistema al usuario. También es conocida como interfaz gráfica y debe tener la característica de ser «amigable» (entendible y fácil de usar) para el usuario. Esta capa se comunica únicamente con la capa de negocio., (Para capturar la información del usuario realiza una comprobación de que no hay errores de formato).

- **Capa de datos:**

Aquí residen los datos, y es la encargada de acceder a los mismos. Está formada por uno o más gestores de bases de datos que, reciben solicitudes de almacenamiento o recuperación de información desde la capa de negocio.

Con esta clasificación, conseguimos que sea más sencillo y mantenible crear diferentes interfaces sobre un mismo sistema, sin requerirse cambio alguno en la capa de datos o lógica.

Ofrece la ventaja de que el desarrollo se puede llevar a cabo en varios niveles y, en caso de que sobrevenga algún cambio, solo afectará al nivel requerido sin tener que revisar entre el código fuente de otros módulos, dado que se habrá reducido el Acoplamiento informático hasta una interfaz de paso de mensajes.



Permite abstraer un nivel del resto, cada grupo de trabajo trabajara en su nivel, de forma que basta con conocer la API que existe entre niveles.

En el diseño de sistemas informáticos actual se suelen usar las arquitecturas multinivel o programación por capas. En dichas arquitecturas a cada nivel se le confía una misión simple, lo que permite el diseño de arquitecturas escalables (que pueden ampliarse con facilidad en caso de que las necesidades aumenten).

Actualmente el diseño más utilizado es en tres niveles (o en tres capas).

Todas estas capas pueden residir en un único ordenador, aunque lo normal es que haya diferentes ordenadores en donde reside la capa de presentación (clientes de la arquitectura cliente/servidor). Las capas de negocio y de datos pueden residir o no en el mismo ordenador, dependiendo de las necesidades según tamaño y complejidad de la base de datos.

Vamos a ver la clasificación más generalizada en el enfoque a nivel de software:

Se divide en:

- Modelo de 2 capas.
- Modelo de 3 capas.
- Modelo de N capas.

Vamos a estudiar cada uno de ellos.

3.6.2.1. Modelo de dos capas

En este modelo, el cliente solicita recursos y el servidor responde directamente a la solicitud con sus propios recursos.

El cliente siempre contendrá la interfaz de usuario.

El servidor no requiere de una aplicación extra para proporcionar el servicio, y siempre se encarga del nivel de datos.

Dependiendo de quién realice las funciones de la lógica de negocio, tendremos un tipo u otro.

El cliente se comunica directamente con un servidor de bases de datos. La aplicación o lógica de negocio bien reside en el cliente, o en el servidor de base de datos en la forma de procedimientos almacenados.

- **Cliente grueso:**
 - Inicialmente, en el modelo de dos capas intervienen equipos que no tienen la característica de mainframe (un servidor de archivos en red) y un cliente "grueso" inteligente, donde se hace la mayor parte del procesamiento.



- Esta configuración no es fácilmente escalable en sistemas de gran, e incluso medio, tamaño (50 o más clientes conectados).
- Entonces el Interfaz Gráfico de Usuario (GUI, Graphical User Interface) emerge como el entorno dominante para las aplicaciones de escritorio y con él, emerge un nuevo enfoque en el planteamiento inicial de la arquitectura de dos capas.
 - » El servidor de ficheros en red de propósito general se reemplaza por un servidor de bases de datos especializado.
 - » Este modelo origina la aparición de nuevas herramientas de desarrollo: PowerBuilder, Visual Basic y Delphi, por citar algunas.
- La mayor parte del procesamiento tiene lugar aún en los clientes "gruesos", pero ahora la información se hace llegar al cliente utilizando un Lenguaje Estructurado de Consulta (SQL, Structured Query Language) para realizar peticiones al servidor de base de datos, que simplemente informa del resultado de las consultas.
- Cuanto más complicada la aplicación, más "grueso" pasa a ser el cliente y más potente debe ser el hardware que debe soportarlo.
 - » El coste de adecuar la tecnología del cliente pasa a ser prohibitivo y puede frustrar la abordabilidad de las aplicaciones.
- Además, la carga de la red utilizando este tipo de clientes es muy grande, de modo que el ancho efectivo de la red (y por lo tanto del número de usuarios que pueden utilizarla) se reduce.
- **Una configuración alternativa "Cliente fino - Servidor grueso"** es otra aproximación utilizada en la arquitectura de dos capas.
 - » En este caso el cliente invoca procedimientos almacenados en el servidor de base de datos.
 - » El modelo del Servidor "grueso" tiene un mejor rendimiento "grueso" porque, aunque la carga de red es todavía pesada, es más ligera que en la aproximación del Cliente "grueso".
 - » Puede ser implementada en un único equipo.
- El inconveniente de esta aproximación es que el uso de procedimientos almacenados hace depender el desarrollo excesivamente del software del vendedor.
- Otro inconveniente se deriva del hecho de que los procedimientos están almacenados conjuntamente con los datos y cada base de datos que contiene el procedimiento debe modificarse cuando cambia la lógica de la aplicación.
 - » En grandes bases de datos distribuidas esto puede conducir a una administración dificultosa.



- En ambos casos, se utiliza un protocolo de transporte de bases de datos (como SQL-net) para llevar las transacciones de un extremo a otro, que generalmente resulta ser un proceso "pesado".
- No importa qué modelo particular se utilice, los sistemas de dos capas no se ajustan bien cuando se manejan aproximadamente 100 usuarios.

En el reparto de funciones en la arquitectura cliente-servidor, está la **Lógica de negocio o Lógica de la aplicación**.

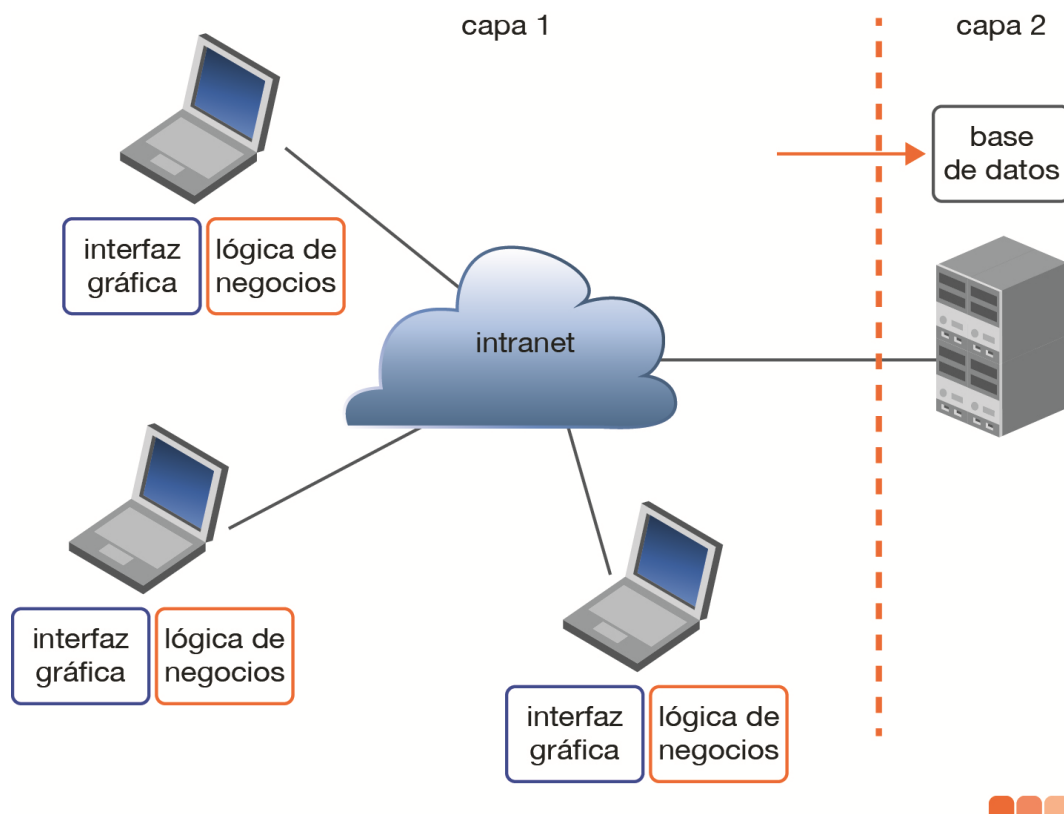
Son funciones que transforman entradas en salidas, incluyendo desde simples sumas hasta complejos modelos matemáticos, financieros, científicos, de ingeniería, etc.

- **Lógica de negocios en el cliente:**

En este esquema el cliente envía mensajes con solicitudes SQL al servidor de bases de datos y el resultado de cada instrucción SQL es devuelto por el servidor.

El cliente se encarga de procesar los registros devueltos según los requisitos que él mismo formuló.

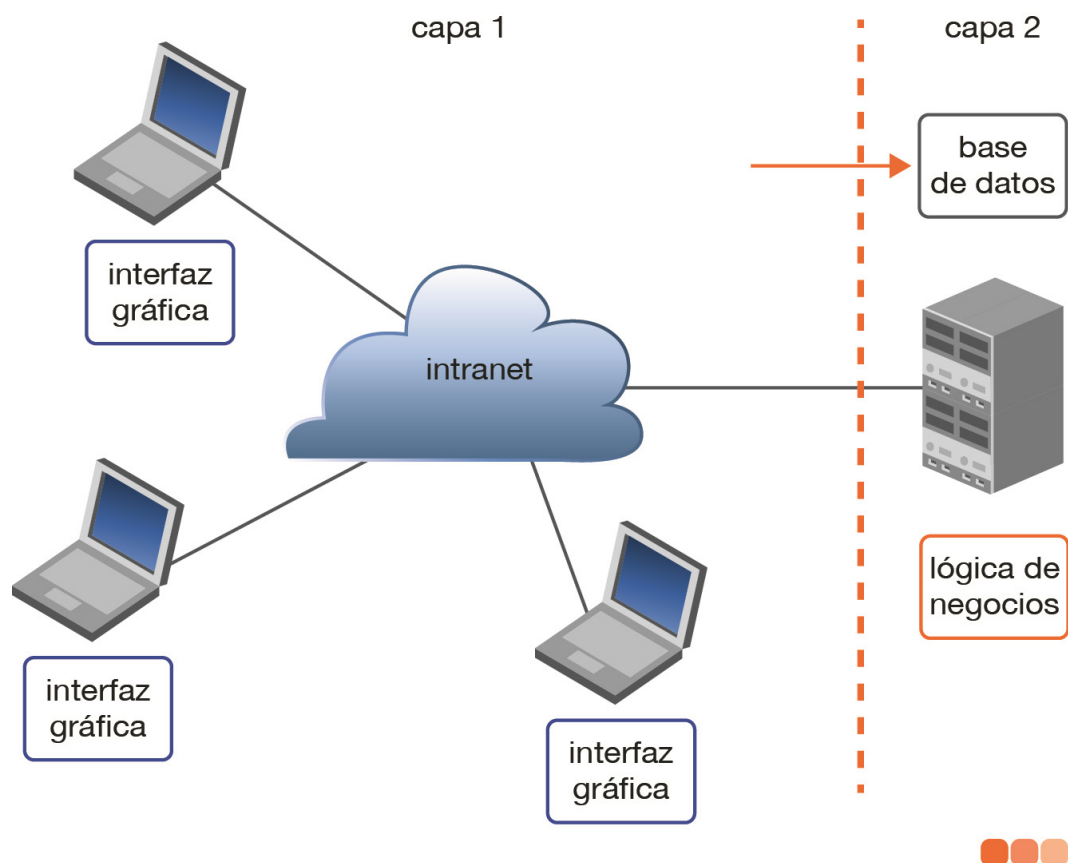
Esta estructura resulta útil en sistemas de apoyo a la decisión y de gestión, pero son inadecuados para sistemas críticos.





- **Lógica de negocios en el servidor:**

En este esquema el cliente envía llamadas a funciones que residen en la base de datos y es esta quien resuelve y procesa la totalidad de las instrucciones SQL agrupadas en la mencionada función.

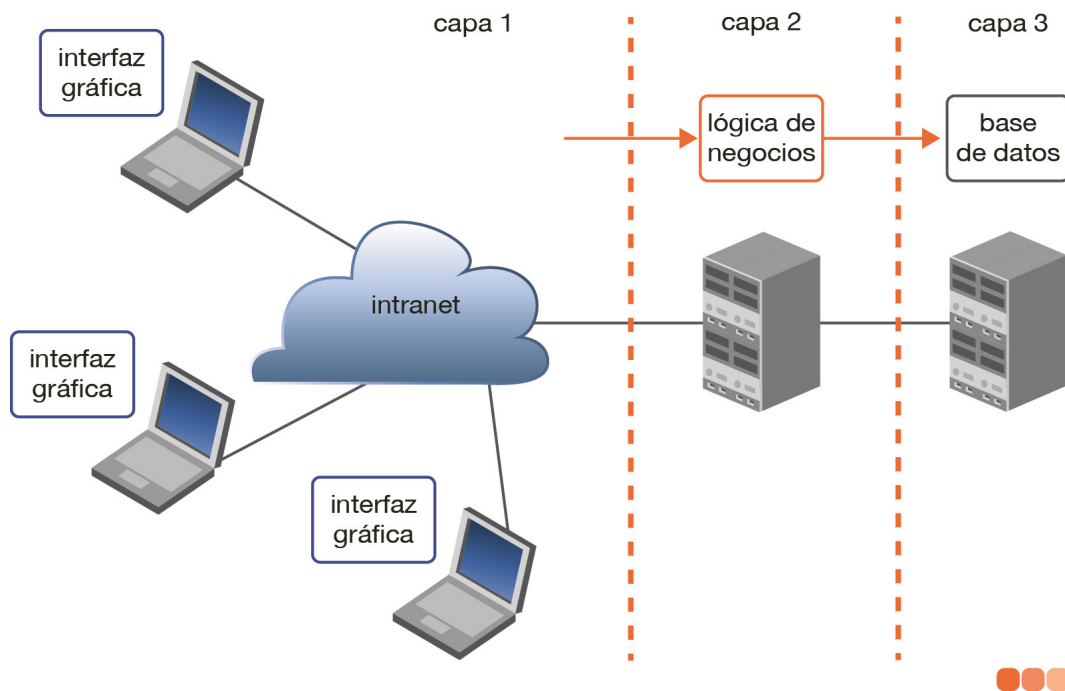


3.6.2.2. Modelo de tres capas

Arquitectura cliente/servidor de tres capas.

Esta estructura se caracteriza por elaborar la aplicación basándose en tres niveles:

- **Capa 1**, lógica de presentación: interfaz.
- **Capa 2**, lógica de negocio.
- **Capa 3**, lógica de Datos o de persistencia.



Características

- Una generación más moderna de la arquitectura C/S añade una capa intermedia (middle tier).
- En la arquitectura de tres capas (en general, en la arquitectura multicapa) el cliente implementa la lógica de presentación (cliente "fino"), el servidor(es) de aplicación implementan la lógica de negocio y los datos residen en uno (o varios) servidor(es) de bases de datos.
- Una arquitectura multicapa se define por tanto por las siguientes tres capas de componentes:
 - Un componente front-end que es el responsable de proporcionar la lógica de presentación.
 - Un componente back-end que proporciona acceso a servicios dedicados, tales como un servidor de bases de datos.
 - Un componente que hace las funciones de capa intermediaria (middle-tier) que permite a los usuarios compartir y controlar la lógica de negocio mediante su aislamiento de la aplicación real.
- Una arquitectura multicapa aumenta la arquitectura C/S tradicional mediante la introducción de una o más componentes intermedios.
 - El sistema cliente interactúa con la capa intermedia vía un protocolo estándar como HTTP o RPC.
 - La capa intermedia interactúa con el servidor de datos (back-end) mediante protocolos de bases de datos estándar tales como SQL, ODBC y JDBC.



- Esta capa intermedia contiene la mayor parte de la lógica de la aplicación, traduciendo las llamadas del cliente en consultas (u otras acciones) a la base de datos y traduciendo los datos provenientes de la base de datos en datos del cliente para devolvérselos.
- Este emplazamiento de la lógica de negocio sobre el servidor de aplicaciones proporciona escalabilidad y aislamiento de la lógica de negocio con el fin de manejar rápidamente los cambios necesarios de ésta.
 - Además, este hecho permite ampliar las opciones en lo que se refiere a la elección de un software propietario de bases de datos.
- La arquitectura de 3 capas se puede extender a n capas cuando la capa intermedia soporta conexiones a diferentes tipos de servicios (no sólo servicios de almacenamiento de datos), integrándolos y acoplándolos al cliente y entre ellos.

Ventajas Principales

- Aporta mayor flexibilidad de desarrollo y de elección de plataformas sobre la cual montar las aplicaciones.
- Provee escalabilidad horizontal y vertical.
- Se mantiene la independencia entre el código de la aplicación (reglas de negocio) y los datos, mejorando la portabilidad de las aplicaciones.
- Permite construir sistemas críticos de alta fiabilidad.
- El mantenimiento es más sencillo.
- Disminuye el número de usuarios conectados a la base de datos.

Otras ventajas de la arquitectura C/S multicapa son:

- Cambios en la interfaz de usuario o en la lógica de la aplicación son muy independientes entre sí, permitiendo a la aplicación evolucionar fácilmente para satisfacer los nuevos requisitos.
- Los cuellos de botella de la red de comunicaciones se minimizan porque la capa de aplicación no transmite datos extras al cliente, sólo lo que necesite para llevar a cabo la tarea.
- Cuando se requieren cambios en la lógica de negocio, sólo debe actualizarse el servidor. En la arquitectura de dos capas, cada cliente debe ser modificado cuando cambia la lógica.
- El cliente está aislado de la base de datos y las operaciones de red. El cliente puede acceder fácil y rápidamente sin saber dónde están los datos o cuántos servidores se están utilizando.
- Las conexiones de bases de datos se pueden agrupar y, por tanto, compartidas por varios usuarios, lo que reduce considerablemente el coste asociado a las licencias por usuario.



- La organización es independiente de la base de datos, porque la capa de datos se escribe utilizando SQL estándar que es independiente de la plataforma.
- La lógica de la aplicación se puede utilizar un lenguaje estándar como Java, C o COBOL.

Inconvenientes

- Dependiendo de la elección de los lenguajes de desarrollo, puede presentar mayor complejidad en comparación con el uso de dos niveles.
- Los ambientes de tres capas pueden incrementar el tráfico en la red.

3.6.2.3. Modelo de N capas (N-Layer)

Se realiza una distribución jerárquica de las responsabilidades y de los roles, para obtener el objetivo de proporcionar una división efectiva de los problemas a resolver.

Las responsabilidades indican la funcionalidad que implementan.

Los roles indican el tipo y la forma de la interacción con otras capas.

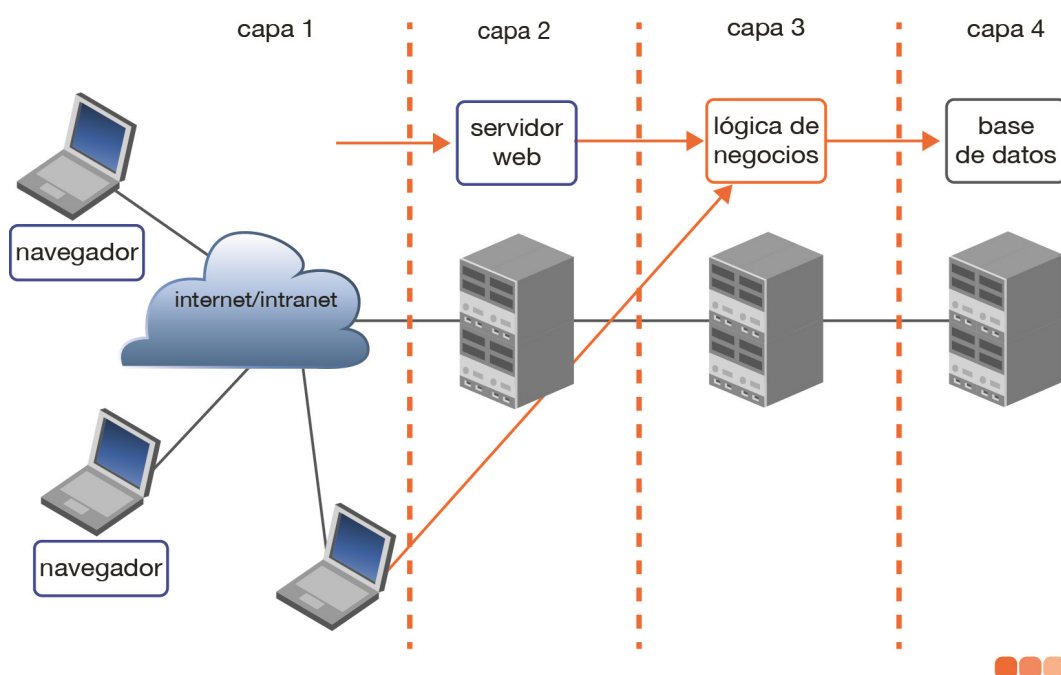
Características

- La funcionalidad de cada capa está claramente separada, contiene solo las tareas de esa capa.
- Las capas de una aplicación, pueden estar en la misma máquina o distribuidas entre varios equipos.
- Interacciones entre capas vecinas: se descomponen los servicios para intentar que las interacciones mayoritariamente sean de esta forma (entre capas vecinas).
- Se utilizan interfaces bien conocidos para la comunicación de los componentes entre capas.
- La comunicación entre capas está basada en una abstracción que proporciona un bajo acoplamiento entre capas.
- Las capas inferiores no tienen dependencias de las capas superiores.
- Cada nivel agrega las responsabilidades y abstracciones del nivel inferior.
- Mostrando una vista completa del modelo, proporciona detalles suficientes para entender las relaciones entre capas.
- No realiza ninguna suposición sobre los tipos de datos, métodos, propiedades y sus implementaciones.



Beneficios

- **Abstracción** (al realizar los cambios a alto nivel se puede incrementar o reducir el nivel de abstracción usado en cada capa).
- **Aislamiento** (realizar actualizaciones en el interior de las capas sin que esto afecte al resto del sistema).
- **Rendimiento** (al distribuir capas en distintos niveles físicos se mejora la escalabilidad, tolerancia a fallos).
- **Testeabilidad** (cada capa tiene una interfaz bien definida sobre la que realizar las pruebas).
- **Independencia** (no se consideran el hardware ni las dependencias con interfaces externas).



4. Arquitecturas de servicios web

Este tipo de arquitectura se basa en desarrollar aplicaciones que se componen de una serie de servicios (componentes) que son reutilizables, y que pueden encontrarse distribuidos en diferentes ordenadores conectados en red.

Podemos llamar a estos servicios desde cualquier lugar (Internet o Intranet) sin importar la plataforma o el lenguaje de programación que tengamos. Y es necesario, utilizar para la comunicación un conjunto de protocolos y estándares (para intercambiar datos entre aplicaciones).

Una Arquitectura de Servicios Web, resumiendo, es una tecnología que utiliza un conjunto de protocolos y estándares que sirven para intercambiar datos entre aplicaciones.



En la arquitectura de servicios web existen tres partes:

- El que pide el servicio web.
- El proveedor de servicios web.
- El publicador.

El que pide el servicio contacta con el publicador y descubre quién es el proveedor (protocolo WSDL) y contacta con el proveedor (protocolo SOAP).

El proveedor valida la petición de servicio y envía el dato estructurado en formato XML utilizando el protocolo SOAP.

El fichero XML es validado de nuevo por el que pide el servicio utilizando un fichero XSD.

Normalmente se utiliza XML para el intercambio de información.

Los servicios web ofrecen información con un formato estándar que puede ser entendido fácilmente por una aplicación.

Así pues, distintas aplicaciones de software desarrolladas en lenguajes de programación diferentes, y ejecutadas sobre cualquier plataforma, pueden utilizar los servicios web para intercambiar datos en redes de ordenadores como Internet.

El objetivo de la información devuelta es que pueda ser fácilmente utilizable por la aplicación que requiere el servicio. (Esto difiere de la web normal, donde las páginas se devuelven en un formato que el usuario puede interpretar fácilmente, pero no así la aplicación).

La interoperabilidad se consigue mediante la adopción de estándares abiertos.

La interoperabilidad es la capacidad de los sistemas de información y de los procedimientos a los que éstos dan soporte, de compartir datos y posibilitar el intercambio de información y conocimiento entre ellos.



Atención

Para mejorar la interoperabilidad entre distintas implementaciones de servicios Web se ha creado el organismo WS-I, encargado de desarrollar diversos perfiles para definir de manera más exhaustiva estos estándares. Es una máquina que atiende las peticiones de los clientes web y les envía los recursos solicitados.



Se han creado estándares comunes de servicios web, lo cual permite crear aplicaciones mediante la integración de servicios indistintamente de la plataforma y el lenguaje de programación utilizado en cada servicio.

Las organizaciones OASIS y W3C son los comités responsables de la arquitectura y reglamentación de los servicios Web.

El W3C (World Wide Web Consortium) define un servicio web como:

Un servicio web es un sistema software diseñado para soportar la interacción máquina-a-máquina, a través de una red, de forma interoperable.

(Estudiaras el W3C en la unidad 9).

Cuenta con una interfaz descrita en un formato procesable por un equipo informático (específicamente WSDL), a través de la que es posible interactuar con el mismo mediante el intercambio de mensajes SOAP, típicamente transmitidos usando serialización XML sobre HTTP conjuntamente con otros estándares web.

4.1. Servicios web

Un servicio web, en inglés, web service (WS) o web services, es un componente al que podemos acceder mediante protocolos Web estándar, utilizando XML para el intercambio de información.

Los Servicios Web permiten distribuir una aplicación a través de Internet, pudiendo utilizar los servicios ofrecidos por cualquier servidor conectado a Internet, por ello es imprescindible la interoperabilidad entre las aplicaciones.

Con lo ya definido, podemos decir de otro modo, que los servicios web proporcionan una forma estándar de interoperar entre aplicaciones software que se ejecutan en diferentes plataformas.

A nivel conceptual, un servicio web es:

- Un componente *software*.
- Proporcionado a través de un *end-point*.

Un *end-point* es un punto de acceso a un servicio.

Si dos servicios interactúan, debemos tener un *end-point* por cada servicio (uno para el que ofrece el servicio y otro para el que lo consume).



Cuando hablamos de servicios web, un *end-point* se identifica mediante una URI.

URI (identificador de recursos uniforme) es una cadena de caracteres que identifica recursos de forma unívoca.

- Accesible a través de la red.

Los servicios productores y consumidores utilizan mensajes para intercambiar información entre ellos en forma de documentos, en los cuales apenas se habla de las capacidades tecnológicas de cada uno de los receptores.



+ Info

Un *end-point* es un punto de acceso a un servicio.

Si dos servicios interactúan, debemos tener un *end-point* por cada servicio (uno para el que ofrece el servicio y otro para el que lo consume).

Cuando hablamos de servicios web, un *end-point* se identifica mediante una URI.

Los servicios web pueden combinarse con muy bajo acoplamiento para conseguir la realización de operaciones complejas.

Ejemplo:



Fuente:

https://es.m.wikipedia.org/wiki/Archivo:AmazonWebservices_Logo.svg



Razones para crear los servicios WEB

La razón principal es que se pueden utilizar con HTTP sobre Transmission Control Protocol (TCP) en el puerto de red 80.

Puesto que, las organizaciones para proteger sus redes utilizan firewalls (filtrando y bloqueando gran parte del tráfico de Internet), que cierran casi todos los puertos TCP salvo el 80, que es el que usan los navegadores web.

Los servicios web se pueden utilizar sobre cualquier protocolo, pero normalmente se utiliza TCP.

(Estudiarás los protocolos en unidades posteriores y en el Bloque IV "Sistemas y Comunicaciones").



+ Info

Otro motivo, es que, antes de SOAP, no había buenas interfaces para acceder a funcionalidades en red.

Las que había eran "ad hoc" y poco conocidas, como: Electronic Data Interchange (EDI), Remote Procedure Call (RPC), u otras API.

Requisitos de los servicios web

Las características deseables de un servicio web son:

- **Interoperabilidad:** un servicio remoto debe permitir su utilización por clientes de otras plataformas.
- **Interfaces fuertemente tipadas:** no debe haber ambigüedad acerca del tipo de dato enviado y recibido desde un servicio remoto, y deben parecerse, en la medida de lo posible, a los tipos genéricos de los distintos lenguajes de programación.
- **Aprovechar los estándares existentes de Internet:** la implementación del servicio remoto debería aprovechar estándares de Internet existentes para evitar *reinventar la rueda*.

Además, puede beneficiarse de su consolidación y de productos creados para dicha tecnología.

- **Soporte para cualquier lenguaje:** la solución no debería ligarse a un lenguaje de programación particular. Un cliente debe ser capaz de implementar un nuevo servicio web existente independientemente del lenguaje de programación en el que se haya escrito tanto el servicio como la aplicación del cliente.



- **Soporte para cualquier infraestructura distribuida:** no debe estar fuertemente ligada a una infraestructura de componentes en particular. Ni siquiera debería requerir tener una infraestructura de objetos distribuidos.

Los protocolos subyacentes deberían proporcionar un nivel base de comunicación entre infraestructura de objeto distribuidos existentes tales como DCOM y CORBA.

- **Un servicio debe poder ser accesible a través de la web:** para ello debe utilizar protocolos de transporte estándares como HTTP, y codificar los mensajes en un lenguaje estándar.
- **Identificación:** un servicio debe contener una descripción de sí mismo. El objetivo es que cualquier aplicación pueda saber su función, su interfaz para poder ser utilizado de forma automática, sin la intervención del usuario.
- **Debe poder ser localizado:** deberemos tener algún mecanismo que nos permita encontrar un servicio web que realice una determinada función.

4.2. Protocolos Web (Web Services Protocol Stack)

BPEL4WS			procesos de negocio
WSDL, UDDI, inspección			descripción
seguridad	intercambio de mensajes de forma confiable	transacciones	calidad de servicio
		coordinación	
SOAP		otros protocolos otros servicios	transporte y codificación
XML, codificación			



La Pila de protocolos para Servicios Web es una colección de protocolos y estándares para redes de Computadores que son utilizados para definir, localizar, implementar y hacer que un Servicio Web interactúe con otro.

Es el conjunto de servicios y protocolos de los servicios web; una colección de protocolos y estándares para redes, que se utilizan para definir, localizar, implementar y hacer que un Servicio Web interactúe con otro.



La Pila de Protocolos está comprendida principalmente por cuatro áreas:

- **Servicio de Transporte:** protocolos responsables del transporte de mensajes entre las Aplicaciones de red.

Algunos de ellos son: HTTP, SMTP, FTP, y el más reciente Blocks Extensible Exchange Protocol (BEEP).

- **Mensajería XML:** se encargan de la codificación de mensajes en un formato común XML, para que puedan ser entendidos en cualquier extremo de una conexión de red.

Algunos de ellos son: XML-RPC, SOAP y REST.

- **Descripción del Servicio:** se usan para describir la interfaz pública de un Servicio Web específico.

El formato de interfaz típicamente usado para esto es el WSDL (Web Services Description Language).

- **Descubrimiento de servicios:** se encarga de centralizar servicios en un registro común, de forma que los servicios Web de la red puedan publicar su localización y descripción, y que sea fácil descubrir que servicios están disponibles en la red.

Actualmente, se utiliza normalmente la API UDDI (Universal Description Discovery and Integration).



+ Info

La Pila de Protocolos para servicios también incluye un amplio rango de protocolos recientemente definidos:

- Business Process Execution Language – BPEL.
- SOAP Security Extensions: Digital Signature – SOAP-DSIG.

4.2.1. XML (Extensible Markup Language)

Formato estándar para los datos que se vayan a intercambiar.

Lo estudiarás en la siguiente unidad "Aplicaciones Web".



4.2.2. SOAP

SOAP, siglas de *Simple Object Access Protocol*. (Los servicios web SOAP, también se denominan servicios web grandes).

Es un protocolo estándar que define, cómo dos objetos en diferentes procesos, pueden comunicarse por medio de intercambio de datos XML.

Se trata de un protocolo derivado de XML que nos sirve para intercambiar información entre aplicaciones.



Anécdota

Este protocolo deriva de un protocolo creado por Dave Winer en 1998, llamado XML-RPC. SOAP fue creado por Microsoft, IBM y otros.

SOAP es un paradigma de mensajería de una dirección sin estado, (sin estado significa que cada mensaje HTTP contiene toda la información necesaria para comprender la petición) que puede ser utilizado para formar protocolos más completos y complejos según las necesidades de las aplicaciones que lo implementan.

Puede formar y construir la capa base de una "pila de protocolos de web service", ofreciendo un framework de mensajería básica en el cual los web services se pueden construir.



Atención

SOAP es una recomendación del W3C.

Está bajo el patrocinio de la W3C El Consorcio WWW, en inglés: World Wide Web Consortium (W3C), que es, un consorcio internacional que genera recomendaciones y estándares que aseguran el crecimiento de la World Wide Web a largo plazo.



Normalmente contienen una descripción legible por la máquina de la descripción de las operaciones ofrecidas por el servicio, escrita en WSDL (*Web Services Description Language*), que es un lenguaje basado en XML para definir las interfaces sintácticamente.

El diseño de un servicio basado en SOAP debe establecer un contrato formal para describir la interfaz que ofrece el servicio web.

Los servicios Web SOAP utilizan mensajes XML para intercomunicarse que siguen el estándar SOAP (*simple object access protocol*), un lenguaje XML que define la arquitectura y formato de los mensajes.

Los lenguajes que cumplen el estándar WSDL ("**Web Service Description Language**"), basados en XML, contienen una descripción de las operaciones ofrecidas por el servicio, las acciones soportadas, que son legibles por la máquina.

Un documento WSDL (*Web Services Description Language*) es un esquema que describe un servicio web, incluida información sobre cómo localizar ese servicio y las operaciones a las que se da soporte.

WSDL puede utilizarse para describir los detalles del contrato, que pueden incluir mensajes, operaciones, *binding* (especifica los protocolos de comunicación usados), y la localización del servicio web. También deben tenerse en cuenta los requisitos no funcionales. Por ejemplo:

- Transacciones.
- Necesidad de mantener el estado (*addressing-direccionamiento*).
- Seguridad.
- Coordinación.

4.2.2.1. Características

El protocolo SOAP tiene tres características principales:

- **Extensibilidad** (seguridad y WS-routing son extensiones aplicadas en el desarrollo).
- **Neutralidad** (bajo protocolo de transporte TCP puede ser utilizado sobre cualquier protocolo de aplicación como HTTP, SMTP o JMS).
- **Independencia** (permite cualquier modelo de programación).

Como ejemplo de cómo el modelo SOAP pueda ser utilizado, consideraremos un mensaje SOAP que podría ser enviado a un web service para realizar la búsqueda de algún precio en una base de datos, indicando para ello los parámetros necesitados en la consulta. El servicio podría retornar un documento en formato XML con el resultado, un ejemplo, precios, localización o características. Teniendo los datos de respuesta en un formato estandarizado procesable (en inglés "parsable"), éste puede ser integrado directamente en un sitio Web o aplicación externa.



La arquitectura SOAP está formada por varias capas de especificación:

- MEP (Message Exchange Patterns) para el formato del mensaje.
- Enlaces subyacentes del protocolo de transporte.
- El modelo de procesamiento de mensajes.
- La capa de extensibilidad del protocolo.

También incluye:

- Conjunto de reglas de codificación para expresar instancias de tipos de datos.
- La Convención para representar llamadas a procedimientos y respuestas.



+ Info

SOAP es el sucesor de XML-RPC, a pesar de que toma el transporte y la neutralidad de la interacción, así como el envelope / header / body, de otros modelos.

4.2.2.2. Mensajes SOAP

Podemos distinguir dos tipos de mensajes según su contenido:

- **Mensajes orientados al documento:** contienen contenido de cualquier tipo que se quiera enviar entre aplicaciones.
- **Mensajes orientados a RPC: PC (Remote Procedure Call): Llamada a procedimiento remoto.**

Es un programa que se utiliza en un ordenador para ejecutar código que se encuentra en otra máquina remota, sin tener que preocuparse por las comunicaciones entre ambas.

De esta manera el programador no tiene que estar pendiente de las comunicaciones, estando estas encapsuladas dentro de las RPC. (Los mensajes invocan procedimientos de forma remota).

Las RPC son muy utilizadas dentro de la comunicación cliente-servidor, es el cliente el que inicia el proceso solicitando al servidor que ejecute cierto procedimiento o función, y el servidor envía de vuelta el resultado de dicha operación al cliente.

En este caso, el contenido del mensaje contendrá el método que queremos invocar junto a los parámetros que le pasamos. El servidor nos deberá devolver un mensaje SOAP con el resultado.

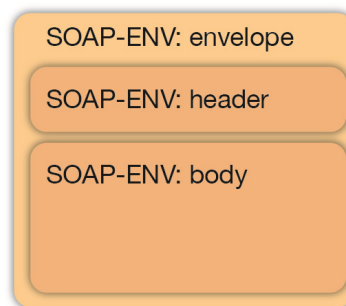
Puede ser utilizado sobre varios protocolos de transporte, aunque está especialmente diseñado para trabajar sobre HTTP.



4.2.2.3. Estructura del mensaje orientado al documento

Un mensaje SOAP es un documento XML ordinario con una estructura definida en la especificación del protocolo. Dicha estructura la conforman las siguientes partes:

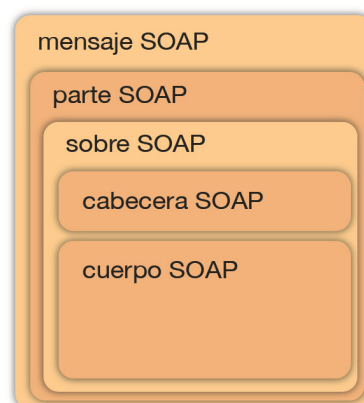
- **Envelope (obligatoria):** representa al elemento que debe aparecer en todo mensaje SOAP, es la parte que identifica al mensaje SOAP como tal.
- **Header:** esta parte es un mecanismo de extensión ya que permite enviar información relativa a cómo debe ser procesado el mensaje. Es una herramienta para que los mensajes puedan ser enviados de la forma más conveniente para las aplicaciones. El elemento "Header" se compone a su vez de "Header Blocks" que delimitan las unidades de información necesarias para el header.
- **Body (obligatoria):** contiene la información relativa a la llamada y la respuesta.
- **Fault:** bloque que contiene información relativa a errores que se hayan producido durante el procesamiento del mensaje y el envío desde el "SOAP Sender" hasta el "Ultimate SOAP Receiver".



Estructura de un mensaje SOAP

Elementos dentro del mensaje SOAP

Dentro del mensaje SOAP podemos distinguir los siguientes elementos:



Elementos de un mensaje SOAP



- **Sobre (*envelope*):**

Incluye:

- El mensaje.
 - A quién va dirigido.
 - Cómo debe ser procesado.
 - Las definiciones de tipos que se usarán en el documento.
 - Una cabecera (opcional).
 - El cuerpo del mensaje.
- **Cabecera (*header*)** opcional: contiene información sobre el mensaje.

Por ejemplo:

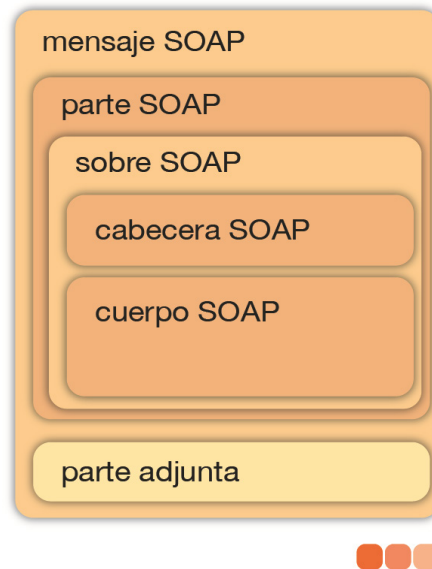
- Por donde ha pasado el mensaje.
 - Podemos especificar si el mensaje es obligatorio (debe ser entendido de forma obligatoria por el destinatario).
 - Indicar los lugares por donde ha pasado el mensaje (actores).
- **El cuerpo del mensaje (*body*):** contiene el mensaje en sí. En el caso de los mensajes RPC, se define una convención sobre cómo debe ser este contenido, en el que se especificará el método al que se invoca y los valores que se pasan como parámetros.

De forma opcional, el cuerpo del mensaje puede contener un error:

- **Error (*Fault*):** servirá para indicar en una respuesta SOAP que ha habido un error en el procesamiento del mensaje de petición que mandamos.

Además de estos elementos, podemos añadir uno más, la especificación de mensajes SOAP con anexos.

Anexo (*Attachment*): nos permite enviar en el mensaje datos que no son XML, como puede ser una imagen. En ese caso tendremos que recurrir a.



Elementos de un mensaje SOAP con anexos

Nuestro mensaje podrá contener tantos anexos como queramos.

XML- binary Optimized Packaging (XOP)

XOP es una especificación añadida al protocolo SOAP en su versión 1.2. Una especificación que aparece para mejorar la eficiencia de la transmisión y la gestión de los datos binarios en mensajes SOAP.

Con XOP se separan los archivos binarios adjuntos (imagen, documento PDF, etc.) del mensaje SOAP principal. Si un Servicio Web envía archivos binarios adjuntos, se servirá de la etiqueta `<xop:Include xmlns:xop="http://www.w3.org/2004/08/xop/include" href="cid:[nombre_del_binario_adjunto]"/>` insertada en el cuerpo del mensaje SOAP (`<Body>`) para referenciar esos archivos que viajarán de manera separada mediante el protocolo MIME (Multipurpose Internet Mail Extensions).

Hasta la aparición de XOP un mismo archivo binario se incrustaba pudiendo enviarse varias veces en el mismo mensaje aumentando así la redundancia de datos y la falta de eficiencia. Con la nueva especificación el mensaje SOAP ya no incluirá los datos binarios, sino que incluirá de contenerlos, las citas o referencias a esos datos binarios.

El paquete MIME que contiene los datos binarios referenciados en el mensaje SOAP es lo que se denomina XOP-Package.



Atención

Axis2: Es un motor nuclear para servicios web.

Axis2, no solo provee la capacidad de agregar servicios web a las aplicaciones web, sino que, además, puede funcionar como servidor autónomo.

Es un rediseño total y una reimplementación completa de la difundida pila SOAP "**Apache Axis**". Existen implementaciones de Axis2 en Java y en C.

4.2.3. UDDI (Universal Description, Discovery and Integration)

Protocolo para publicar la información de los servicios web. Permite comprobar qué servicios web están disponibles.

UDDI es un estándar XML para describir, publicar y encontrar (localizar) servicios web.

UDDI, es un directorio de interfaces de servicios web descritos en WSDL que se comunican mediante SOAP, que se encarga de dar a conocer el servicio web definido, para que los clientes interesados sepan de su existencia y puedan utilizarlo en sus aplicaciones. Así las compañías pueden registrar y buscar servicios web.

UDDI es una iniciativa del sector para hacer compatible el descubrimiento de servicios Web con todo tipo de tecnologías y plataformas.

Propiedades de UDDI

- UDDI nos permite localizar servicios web.
- Define la especificación para construir un directorio distribuido de servicios web, donde los datos se almacenan en XML.
- Es un protocolo basado en SOAP que define cómo se comunican los clientes con los registros UDDI.
- Es un conjunto de registros duplicados globales en particular.



UDDI incluye un esquema XML para mensajes SOAP que define un conjunto de documentos para describir información de empresas y servicios, un conjunto común de API para consultar y publicar información en los directorios y una API para duplicar entradas de directorio entre nodos UDDI iguales.

En estos registros se almacena información sobre:

- Los servicios.
 - Las organizaciones que los proporcionan.
 - La categoría en la que se encuentran.
 - Sus instrucciones de uso (normalmente en WSDL).
- Define una API para trabajar con dicho registro, que nos permitirá buscar datos almacenados en él y publicar datos nuevos.
 - Para acceder al registro se utilizarán mensajes SOAP, transportados mediante el protocolo HTTP.
 - Una aplicación podrá anunciar sus servicios en un registro UDDI, o bien localizar servicios que necesitemos mediante este registro.
 - Esta capacidad de localizar servicios en tiempo de ejecución y de que una aplicación pueda saber cómo utilizarlo inmediatamente, gracias a la descripción del servicio, nos permitirá realizar una integración débilmente acoplada de nuestra aplicación.

Tipos de información en UDDI

Por lo que ya hemos visto, tenemos tres tipos de información relacionados entre sí:

- **Páginas blancas:** datos de las organizaciones (dirección, información de contacto, etcétera).
- **Páginas amarillas:** clasificación de las organizaciones (según tipo de industria, zona geográfica, etcétera).
- **Páginas verdes:** información técnica sobre los servicios que se ofrecen. Aquí se dan las instrucciones para utilizar los servicios. Es recomendable que estas instrucciones se especifiquen de forma estándar mediante un documento WSDL.

Formatos de los registros UDDI

Los registros UDDI tienen dos formatos que se ajustan a las mismas especificaciones:

- **Un registro privado:** que permite publicar, y probar, las aplicaciones e-business internas en entornos privados y seguros.



- **Un registro público:** es una colección de directorios iguales que contienen información sobre empresas y servicios. Localiza servicios que se registran en uno de sus nodos iguales y facilita el descubrimiento de servicios Web publicados.

Se duplican los datos en todos los registros de forma regular. Esto asegura la coherencia en los formatos de descripción de servicios y facilita el seguimiento de los cambios a medida que se producen.



+ Info

IBM® mantiene dos registros públicos llamados IBM UDDI Business Registry e IBM UDDI Test Registry. IBM UDDI Test Registry permite desarrollar el servicio Web y experimentar el proceso de registro UD-DI sin poner el servicio Web en un registro oficial. Utilice IBM UDDI Test Registry para experimentar con UDDI y para probar y validar el servicio Web.

4.2.4. WSIL

WSIL (Web Services Inspection Language) es un lenguaje y formato de descripción utilizado en el contexto de servicios web. WSIL se utiliza para describir y descubrir servicios web disponibles. Su meta es la facilitar información sobre la disponibilidad y ubicación de servicios web para que los clientes y usuarios puedan descubrir y acceder a ellos de manera eficiente. Es un mecanismo de descubrimiento de servicios alternativo y complementario a UDDI. A diferencia de UDDI puede estar distribuido en ubicaciones específicas y es más ligero. WSIL por otro lado, se centra en proporcionar el descubrimiento de servicios web sin necesidad de un registro centralizado como es el caso de UDDI.



¡Pregunta de examen!

En el examen de septiembre de 2023, la corrección provisional da esta respuesta como válida. Pues figura de manera textual en la URL de IBM [WSIL \(Web Services Inspection Language\)](#)



Señale la afirmación correcta con respecto a WSIL:

a) Es un método alternativo al descubrimiento de servicios Web.

La cuestión es que la frase sacada de contexto no parece muy acertada pues entendemos como la imagen siguiente parece indicar que es un método alternativo a UDDI.

Según nuestro parecer la respuesta más correcta sería la c.

c) Define un modo de publicar y encontrar información sobre servicios Web.

WSIL (Web Services Inspection Language)

Última actualización: 2021-03-05

WSIL (Web Services Inspection Language) es un mecanismo de descubrimiento de servicios que es una alternativa a UDDI además de ser complementario a UDDI. Cuando encuentra servicios Web con UDDI, se dirige a un registro centralizado. **WSIL es un método alternativo al descubrimiento de servicios Web.** WSIL le permite dirigirse directamente al proveedor de servicios y solicitar los servicios que proporciona.

4.2.5. WSDL (Web Services Description Language)

Es el lenguaje de la interfaz pública para los servicios web. Es una descripción basada en XML de los requisitos funcionales necesarios para establecer una comunicación con los servicios web.

WSDL es una parte integral del estándar UDDI, y es el lenguaje que éste utiliza.

WSDL es un lenguaje basado en XML utilizado para describir la funcionalidad que proporciona un servicio web, y cómo acceder a ellos.



Atención

Es el formato estándar para describir un web service, y fue diseñado por Microsoft e IBM.

Una descripción WSDL (fichero WSDL) de un servicio web proporciona una descripción de cómo se debe llamar al servicio, qué parámetros espera y qué estructuras de datos devuelve. WSDL describe un servicio utilizando varios elementos (etiquetas XML).

Estructura del WSDL

La estructura del WSDL tiene los siguientes elementos:

- **Tipos de datos** <types>:

Esta sección define los tipos de datos usados en los mensajes. Se utilizan los tipos definidos en la especificación de esquemas XML.

- **Mensajes** <message>:

Aquí definimos los elementos de mensaje. Cada mensaje puede consistir en una serie de partes lógicas. Las partes pueden ser de cualquiera de los tipos definidos en la sección anterior.

- **Tipos de puerto** <portType>:

Con este apartado definimos las operaciones permitidas y los mensajes intercambiados en el Servicio.

- **Bindings** <binding>:

Especificamos los protocolos de comunicación usados.

- **Servicios** <services>:

Conjunto de puertos y dirección de los mismos. Esta parte final hace referencia a lo aportado por las secciones anteriores.



+ Info

Con estos elementos no sabemos qué hace un servicio pero sí disponemos de la información necesaria para interactuar con él (funciones, mensajes de entrada/salida, protocolos...).

Clasificación de los elementos WDSL

Podemos clasificarlos en dos tipos:

- **Elementos abstractos:**

La parte WSDL abstracta describe las operaciones y mensajes con detalle. Es decir, especifica *qué* hace el servicio:

- Qué operaciones están disponibles.
- Qué entradas, salidas, y mensajes de error tienen las operaciones.
- Cuáles son las definiciones de los tipos para los mensajes de entrada, salida y error.

La parte abstracta de un WSDL contiene dos componentes principales:

- Las operaciones que forman la definición de la interfaz.
- Los tipos de datos para los parámetros de entrada, salida y error.

- **Elementos concretos:**

La parte WSDL concreta describe el *cómo* y el *dónde* del servicio:

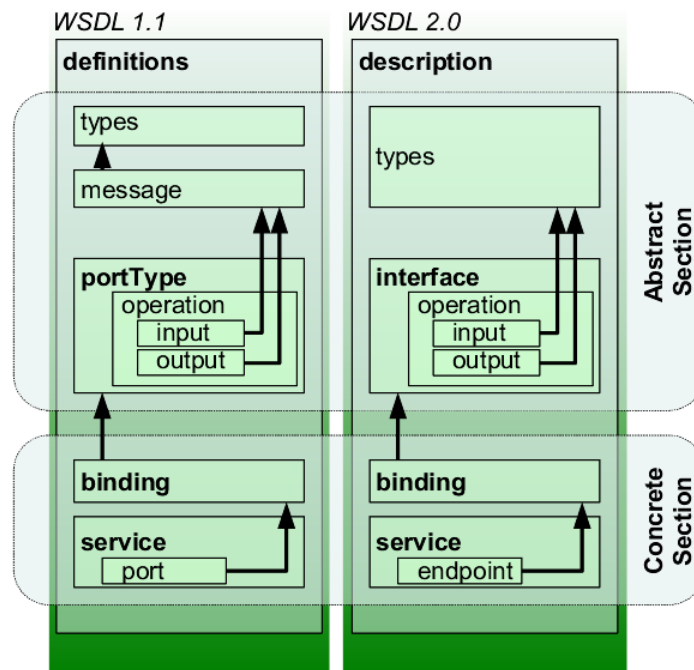
- Cómo tiene que llamar un cliente al servicio.
- Qué protocolo debería usar.
- Dónde está disponible el servicio.

La parte concreta de un WSDL contiene dos componentes principales:

- Información de enlazado (*binding*) sobre el protocolo que se va a utilizar.
- La dirección donde localizar el servicio.



A continuación, mostramos la estructura de los ficheros WSDL en sus dos versiones:



Fuente: https://commons.wikimedia.org/wiki/File:WSDL_11vs20.png

4.2.6. Estándares de Seguridad y Gestión Avanzada en Servicios Web

Estos protocolos están relacionados específicamente con los servicios web y sus aspectos relacionados con la seguridad, la confiabilidad y la gestión de políticas.

WS-Addressing

Especificación de servicios web que proporciona un marco para agregar información de direccionamiento a los mensajes SOAP (Simple Object Access Protocol). Esto permite a las aplicaciones web identificar el remitente y el destinatario de un mensaje y especificar direcciones de respuesta.

WS-Federation

Especificación de servicios web utilizado en la federación de identidades, lo que permite la autenticación única (Single Sign-On) y la autorización entre diferentes sistemas y dominios de seguridad.



WS-Policy

Especificación que permite a las partes en una interacción de servicios web definir y comunicar sus políticas de seguridad, confiabilidad y otros aspectos de la interacción.

WS-ReliableMessaging

Especificación que se centra en la garantía de la entrega confiable y en el orden correcto de los mensajes en servicios web basados en SOAP. Esto es esencial para aplicaciones que requieren comunicaciones seguras y confiables, como transacciones financieras, servicios de salud y aplicaciones críticas. La especificación WS-ReliableMessaging proporciona un conjunto de estándares que los desarrolladores pueden implementar para abordar estos requisitos de confiabilidad.

WS-SecureConversation

Especificación que se utiliza para establecer canales seguros y confiables en una comunicación entre servicios web, lo que incluye la autenticación y la protección de la confidencialidad e integridad de los mensajes. Define los mecanismos para establecer y compartir contextos de seguridad y obtener claves de contextos de seguridad.

WS-Security

Especificación esencial para garantizar la seguridad en servicios web basados en SOAP, proporcionando mecanismos para autenticación, firma digital, cifrado y gestión de tokens de seguridad en las comunicaciones web. Esto es fundamental para proteger la integridad y la confidencialidad de los datos en las transacciones de servicios web.

Entre los servicios públicos que utilizan WS-Security como estándar para proteger sus comunicaciones **SOAP** destacan **GEISER**, **DIR3**, **SIR** y **@firma**. Todos ellos manejan información sensible y requieren firmar digitalmente las peticiones, garantizando así autenticación fuerte, integridad del mensaje y trazabilidad en los intercambios.

GEISER permite el envío de asientos registrales entre administraciones; **DIR3** proporciona el directorio común de unidades administrativas; **SIR** gestiona la interconexión entre oficinas de registro; y **@firma** ofrece servicios de validación de certificados y firma electrónica. En todos los casos, **WS-Security** es clave para proteger sus servicios web.

WS-Trust

Especificación de seguridad esencial en el mundo de los servicios web, que permite la gestión de la confianza y la autenticación de usuarios y aplicaciones a través de la emisión y validación de tokens de seguridad. Esta especificación es ampliamente utilizada en entornos empresariales y en aplicaciones que requieren un alto nivel de seguridad en las transacciones en línea.



4.2.7. REST (Representational State Transfer)

Es una arquitectura que, haciendo uso del protocolo HTTP, proporciona una API que utiliza cada uno de sus métodos (GET, POST, PUT, DELETE, etcétera) para poder realizar diferentes operaciones entre la aplicación que ofrece el servicio web y el cliente.

Además de responder a formatos de intercambio que no sea HTML, (como a Json o XML), REST involucra otros conceptos como:

- Acceder a recursos utilizando un identificador global (URI).
- Operaciones bien definidas para crear, leer, actualizar y eliminar.
- Trabajar bajo un protocolo cliente-servidor sin estado, (como HTTP).



Básico

- REST es una arquitectura que se ejecuta sobre HTTP.
- RESTful se utiliza para referirse a los servicios web que ejecutan la arquitectura REST.

4.2.7.1. RESTful

RESTful (Representational State Transfer Web Services) es servicio web que implementa la arquitectura REST.

Los servicios web RESTful son adecuados para escenarios de integración sencillos *ad-hoc*.



+ Info

Una solución *ad-hoc* es aquella que se crea para un fin específico y por lo tanto no es utilizable para otros propósitos.



RESTful está diseñado para utilizar el protocolo HTTP, ya que estos servicios web se suelen integrar mejor con HTTP que los servicios basado en SOAP, ya que no utiliza mensajes XML o definiciones del servicio en forma de fichero WSDL.

Se utiliza para aplicaciones sencillas. Los servicios web REST utilizan estándares muy conocidos como HTTP, URI, MIME. Tienen una infraestructura *ligera* que permite que los servicios se construyan con poco coste y esfuerzo.

Estos servicios web se suelen integrar mejor con HTTP que los servicios basado en SOAP, ya que no requieren mensajes XML o definiciones del servicio en forma de fichero WSDL.



+ Info

La API de Java JAX-RS, se encarga de la implementación de servicios web Rest o RestFul.

4.2.7.2. API REST

REST es el estándar más lógico, eficiente y habitual en la creación de APIs para servicios de Internet.

Rest se refiere a cualquier interfaz entre sistemas que utilicen HTTP para obtener datos en todos los posibles formatos y realizar operaciones sobre ellos (como hace XML y JSON).

Es más sencillo que SOAP, que dispone de una gran capacidad, pero también es mucho más complejo.

Características de REST:

- **Protocolo cliente/servidor sin estado:**

No es necesario que ni cliente ni servidor recuerden estado previo para satisfacer una petición HTTP, ya que esta petición contiene toda la información para ser ejecutada.

(Aunque no es aconsejable, algunas aplicaciones HTTP incorporan memoria cache, se les denomina protocolo cliente-caché-servidor sin estado). Se definen respuestas concretas a peticiones para que se pueda dar igual respuesta a peticiones idénticas en el futuro).

- **Los métodos que soporta una API REST en HTTP, que realizan las operaciones más importantes sobre los datos, son cuatro:**

- POST: crear.
- GET: leer y consultar.
- PUT: editar.
- DELETE: eliminar.



- En REST, los objetos siempre se manipulan a partir de la URI.

- **Interfaz uniforme:**

Sistematiza el proceso con la información. Siempre aplica los cuatro métodos vistos (post, get, put y delete) sobre los recursos cuando están identificados por una URI.

- **Sistema de capas:**

Cada una de las capas se encarga de una funcionalidad, teniendo una arquitectura jerárquica entre los componentes.

- **Uso de hipermedios:**

Ted Nelson acuñó el término de hipermedio y de hipertexto en 1965.

Un hipermedio, designa al conjunto de métodos o procedimientos para escribir, diseñar o componer contenidos que integren texto, imagen, video, audio, mapas y otros soportes de información emergentes, de tal modo que el resultado obtenido, además, tenga la posibilidad de interactuar con los usuarios.

(Hipermedio es una extensión del concepto de hipertexto).

En API REST, el concepto de hipermedia indica la capacidad que tiene la interfaz proporcionar al cliente y al usuario los enlaces adecuados para ejecutar acciones concretas sobre los datos.

- **Obligatoriedad de HATEOAS:**

Acrónimo de: Hypermedia As The Engine Of Application State, en castellano Hipermedia Como Motor del Estado de la Aplicación.

Disponer del principio HATEOAS, significa que, cada vez que se realiza una petición al servidor, una parte de la respuesta contendrá información de los hipervínculos de navegación asociada a otros recursos del cliente.

Ventajas de REST para el desarrollo:

- **Separación entre cliente y servidor:**

El protocolo REST aísla totalmente la interfaz de usuario, logrando:

- Aplicaciones más flexibles, al facilitar tener en servidores distintos el front y el back.
- Aumentar la escalabilidad de los proyectos.
- Permitir, que el desarrollo de los distintos componentes, evolucionen independientemente (interfaz de usuario, almacenamiento de datos, servidor).



- Facilitar la portabilidad de la interfaz a otro tipo de plataformas.

Siempre que las respuestas a las peticiones se hagan en el lenguaje de intercambio de información usado (normalmente XML o JSON), se puede tener con una API REST servidores PHP, Java, Python o Node.js.



+ Info

GRAPHQL, es una arquitectura alternativa a REST.

4.2.7.3. Documentación de APIs REST

4.2.7.3.1. RAML (RESTful API Modeling Language)

RAML es un lenguaje de definición de APIs basado en YAML que permite modelar APIs REST de forma sencilla, clara y reutilizable. Su propósito principal es la documentación estructurada, permitiendo describir rutas, métodos, parámetros y tipos de datos, facilitando tanto el diseño colaborativo como la generación automática de documentación, pruebas y código. Aunque es más común en el ecosistema de MuleSoft, es una solución abierta y compatible con diversas herramientas.

```
##RAML 1.0
title: API de Discos
version: v1
/users:
  get:
    description: Lista de discos
    responses:
      200:
        body:
          application/json:
            example: |
              [{ "id": 1, "nombre": "Strange Days" }]
```



4.2.7.3.2. OpenAPI / Swagger

OpenAPI (antes Swagger) es una especificación ampliamente adoptada para definir APIs REST. Permite describir recursos, operaciones, parámetros, respuestas y códigos de estado. Su formato (YAML o JSON) es interpretable por herramientas como Swagger UI para mostrar documentación interactiva o Swagger Codegen para generar código cliente o servidor automáticamente. Es una pieza clave en entornos DevOps y arquitectura basada en contratos.

```
openapi: 3.0.0
info:
  title: API de Discos
  version: 1.0.0
paths:
  /usuarios:
    get:
      summary: Lista todos los usuarios
      responses:
        '200':
          description: OK
          content:
            application/json:
              example:
                - id: 2
                  nombre: Waiting for the sun
```

4.2.8. Otros protocolos

4.2.8.1. Protocolos genéricos para transmisión de XML

Los datos en XML también pueden enviarse de una aplicación a otra mediante protocolos normales como Hypertext Transfer Protocol (HTTP), File Transfer Protocol (FTP), o Simple Mail Transfer Protocol (SMTP).



4.2.8.2. Protocolos especializados en XML

XML-RPC

XML-RPC (Remote Procedure Call basado en XML) es un protocolo simple utilizado para la invocación remota de procedimientos a través de la red, en el que los mensajes se codifican utilizando XML. Este protocolo permite que una aplicación o sistema realice una llamada a un procedimiento o función que reside en otro servidor, facilitando la comunicación entre sistemas distribuidos. Al estar basado en XML, XML-RPC permite la interoperabilidad entre plataformas y lenguajes de programación, ya que XML es un formato ampliamente soportado.

El funcionamiento básico de XML-RPC consiste en que el cliente crea un mensaje XML con la solicitud de invocar un procedimiento, que incluye el nombre del método y los parámetros correspondientes. Este mensaje es enviado al servidor a través de HTTP o HTTPS. El servidor procesa la solicitud, ejecuta el procedimiento y responde con un mensaje XML que contiene el resultado de la operación. El cliente recibe la respuesta y la interpreta.

Entre las ventajas de XML-RPC se destacan su simplicidad de implementación y su capacidad para facilitar la interoperabilidad entre sistemas diversos, lo que lo convierte en una opción atractiva para proyectos de integración entre aplicaciones desarrolladas en diferentes lenguajes. Además, como utiliza HTTP, es compatible con la mayoría de los firewalls y proxies, lo que facilita su uso en Internet.

Sin embargo, su principal desventaja es el tamaño de los mensajes, debido al uso de XML, lo que puede afectar el rendimiento, especialmente en aplicaciones con grandes volúmenes de datos o en redes con limitaciones de ancho de banda. A pesar de ser más ligero que otros protocolos como SOAP, XML-RPC no está diseñado para manejar comunicaciones muy complejas o con grandes volúmenes de datos, por lo que en proyectos de mayor escala, se suelen preferir otras alternativas como REST.

5. Arquitectura SOA

SOA, son las siglas de Service Oriented Architecture, traducido en castellano como Arquitectura Orientada a Servicios, es un tipo de diseño de software que permite reutilizar sus elementos.

Las arquitecturas SOA son un modelo orientado a la reutilización de los servicios en entornos de sistemas distribuidos, gracias a las interfaces de servicios que se comunican a través de una red con un lenguaje común.

SOA surge del paradigma orientado a servicios, de su análisis, y posee una serie de principios inviolables y de patrones que definen como SOA se expresa y funciona.

SOA es un marco de trabajo conceptual, un enfoque que permite diseñar y construir soluciones de negocio específicos, las organizaciones, a partir de componentes independientes que exponen funciones como servicios accesibles por otros componentes a través de interfaces estándares, pueden unir los objetivos de negocio con la infraestructura TI, integrando los datos y la lógica de negocio de sus sistemas separados.



Por ejemplo, un resultado de negocio específico es comprobar el crédito de un cliente, obtener datos de clima, consolidar reportes de movimientos sísmicos...



Resumiendo

SOA es una arquitectura de aplicación.

Todas sus funciones están definidas como servicios independientes, con interfaces invocables que pueden ser llamados para formar los procesos de negocio.

Características de la arquitectura SOA

- Basarse en el diseño de servicios que reflejan las actividades del negocio en el mundo real, estas actividades hacen parte de los procesos de negocio de la compañía.
- Representar los servicios utilizando descripciones de negocio para asignarles un contexto de negocio.
- Tener requerimientos de infraestructura específicos y únicos para este tipo de arquitectura, con el uso recomendable de estándares abiertos para la interoperabilidad y transparencia en la ubicación de servicios.
- Estar implementada de acuerdo con las condiciones específicas de la arquitectura de TI en cada compañía.
- Requerir un control fuerte sobre las representación e implementación de servicios.
- Requerir un conjunto de pruebas que determinen que es un buen servicio.

Objetivos de SOA

- Alinear las capacidades de TI con los objetivos de negocio.
- Ofrecer una respuesta rápida y fácil para los cambios de requerimientos a través de su infraestructura.



Atención

El desarrollo e implementación de una arquitectura SOA se rige por los principios descritos en el **Manifiesto SOA**.

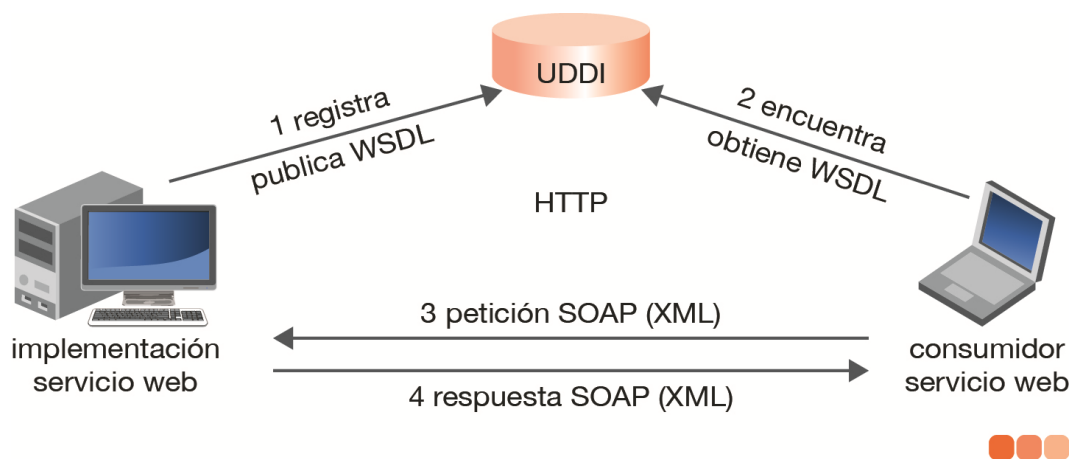
http://www.soa-manifesto.org/default_spanish.html

Roles en SOA

La arquitectura de servicios web (SOA) utiliza los principios y tecnologías básicos de los servicios web, es decir:

- SOAP como lenguaje de intercambio.
- WSDL como lenguaje para la descripción de los servicios.
- UDDI para la publicación o registro de estos.

En la figura puedes observar la estructura básica de funcionamiento de una SOA.



Se puede observar la existencia de tres roles:

- **Cliente del servicio:** es quien solicita la ejecución del servicio web y lo consume.
- **Proveedor del servicio:** es el encargado de implementar el servicio web y ofrecerlo a los clientes.
- **Registro del servicio:** es un repositorio donde se almacenan las descripciones de los servicios. Los clientes podrán usarlo para buscar el servicio web que mejor se adapte a sus necesidades.



Los pasos que hay que seguir son los siguientes:

1. El proveedor del servicio da de alta el servicio web en el registro, almacenando en registro el documento de descripción del servicio.
2. El solicitante del servicio busca en el registro un servicio web que pueda adaptarse a sus necesidades.
3. Una vez seleccionado el servicio, el solicitante lo invoca mediante el envío de un mensaje SOAP, en el cual se indica la acción a realizar y los datos de entrada.
4. El servicio web recibe la petición y ejecuta la funcionalidad.
5. El servicio web envía un mensaje SOAP al solicitante con los resultados obtenidos.



Atención

No es lo mismo SOA que Microservicios.

Los microservicios son también un patrón de arquitectura de software, aunque, en este caso, está construido en base a aplicaciones complejas y los pequeños procesos independientes que las componen, comunicándose entre sí mediante APIs agnósticas del lenguaje.

Diferencias de SOA con MSA (arquitecturas de microservicios)

- **Compartición de componentes:**
 - MSA minimiza el uso compartido de componentes a través de un contexto limitado.
 - SOA le saca todo el partido, algo que, por otra parte, aumenta las latencias, haciendo que los sistemas basados en este tipo de arquitectura sean más lentos.
- **Granularidad de servicio:**
 - MSA se compone de servicios de propósito único y muy especializados.
 - SOA ofrece servicios más versátiles en cuanto a su funcionalidad empresarial.



- **Coordinación:**

- MSA generalmente carecen de coordinación entre sí, o si la presentan es mínima.
- En SOA, es necesario coordinar con varios grupos para atender a las solicitudes de negocio.

- **Middleware:**

- MSA trabajan con una capa de API creada entre los servicios y los consumidores de servicios. No utiliza Middleware.
- SOA utiliza Middleware, ofreciendo así, capacidades adicionales que no se encuentran en MSA, como: mediación y enrutamiento, mejora de mensajes, mensajes y transformación de protocolos.

- **Interoperabilidad heterogénea:**

- MSA es preferible cuando todos los servicios puedan quedar expuestos y se deba usar el mismo protocolo de acceso remoto, ya que MSA intenta simplificar el patrón de arquitectura al reducir el número de opciones de integración.
- SOA promueve la propagación de múltiples protocolos heterogéneos a través de su componente middleware de mensajería, por eso, esta opción debe tenerse en cuenta en los casos en que el objetivo sea lograr la integración de varios sistemas utilizando diferentes protocolos en un entorno heterogéneo.



+ Info

Ensemble de Intersystems es una plataforma flexible para la conectividad rápida y para el desarrollo de nuevas aplicaciones conectables.

Es utilizado por empresas que desean una tecnología de integración avanzada, estableciendo infraestructuras basadas en ESB y SOA.

ESB: el bus de servicios empresariales, modelo de arquitectura de software que gestiona la comunicación entre servicios web.



6. Bibliografía

- TANENBAUM, A. *Sistemas distribuidos: principios y paradigmas*. Editorial Prentice Hall.
- <http://www.sc.ehu.es/acwlaroa/SDI/Apuntes/Cap1.pdf>.
- <http://www.unap.edu.pe/cidiomas/licing/pdf/sd.pdf>.
- <https://oposicionestic.blogspot.com/2011/06/arquitectura-cliente-servidor.html>.
- https://www.ecured.cu/Arquitectura_Cliente_Servidor.
- <http://www.jtech.ua.es/j2ee/publico/servc-web-2012-13/sesion01-apuntes.html>.
- <https://sistemasdistribuidos938.wordpress.com/>.
- <https://www.transponder1200.com/serie-confiabilidad-en-sistemas-de-mision-critica-empezar-por-el-principio/>.
- https://administracionelectronica.gob.es/pae_Home/pae_Estrategias/pae_Interoperabilidad_Inicio.html.
- <https://blog.powerdata.es/el-valor-de-la-gestion-de-datos/que-es-soa-y-cual-es-su-diferencia-con-los-microservicios>.
- <https://desarrolloweb.com/articulos/1589.php>.
- <https://www.disrupciontecnologica.com/capas-y-niveles-diseno-y-confusion/>.
- <https://revistabyte.es/actualidad-it/integracion/solucion-ensemble-intersystems/>.
- https://www.ibm.com/support/knowledgecenter/es/SSBLQQ_9.1.0/com.ibm.rational.rit.protocol.doc/topics/c_rithttp_wsdl_soap_msgs.html.
- <http://uddi.ibm.com>.
- <http://www.uddi.org/register.html>.
- http://laurel.datsi.fi.upm.es/_media/docencia/asignaturas/sod/comunicaciones-parte1-4pp.pdf?id=docencia:asignaturas:sod&cache=cache.
- <https://bbvaopen4u.com/es/actualidad/api-rest-que-es-y-cuales-son-sus-ventajas-en-el-desarrollo-de-proyectos>.
- <https://www.infor.uva.es/>.
- https://es.wikipedia.org/wiki/Servicio_web#Estándares_empleados.



- https://es.wikipedia.org/wiki/Web_Services_Protocol_Stack.
- <https://desarrolloweb.com/manuales/54/>.
- https://www.w3schools.com/xml/xml_services.asp.
- <http://arquitecturaorientadaaservicios.blogspot.com/2006/06/soa-y-los-servicios-web-i.html>.
- https://es.wikipedia.org/wiki/Arquitectura_de_software.
- <https://silo.tips/download/5-arquitectura-de-servicios-web-ws>.
- <https://www.disrupciontecnologica.com/arquitectura-de-servicios-web>.
- <https://www.transponder1200.com/serie-confiabilidad-en-sistemas-de-mision-critica-empezar-por-el-principio/>.